UAS OneSpan Token Implementation Guide - 6.0.1-GA (AI)

Table of Contents

1. Preface - UAS OneSpan Token Implementation Guide

This document describes integrating and configuring the OneSpan (formerly VASCO) token with AccessMatrix Universal Authentication Server (UAS). In addition, it consists of a detailed API integration of the OneSpan token with the AccessMatrix UAS system. It is a reference for the security administrator, professional service and developers.

Intended Reader

This is intended as a reference for the security administrators, i-Sprint's Professional Service personnel, and developers.

Document Scope

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- *AccessMatrix Common Administration Guide*
- *UAS Administration Guide*
- *AccessMatrix REST API Specification*

2. Introduction to UAS OneSpan Token Implementation

This document provides the instructions on the configuration and API integration for the UAS OneSpan Token solution.

About UAS Onespan Token

AccessMatrix UAS system supports the integration of OneSpan OTP tokens to address the strong authentication required for a wide range of enterprise applications. The solution provides administration, authentication, authorization, and audit services to address security and compliance requirements. The following components are required for multi-factor authentication:

- AccessMatrix Universal Authentication Server (UAS)
- OneSpan Digipass Hardware Tokens

Key Benefits

- **Enhanced security with Multi-Factor Authentication**
  Multi-factor authentication provides exceptionally high levels of security to protect against hacking attacks. Two-factor authentication, a type of multi-factor authentication with DIGIPASS authenticators, is used to secure network access, protect financial accounts, digitally sign transactions, and keep criminals from causing losses by preventing unauthorized network access.
- **Ease of use**
  Digipass hardware authenticators are easy to implement and integrate. It is a fast and simple process supported by OneSpan Authentication Server Framework (formerly VACMAN Controller) solution.
- **Convenience of use**
  The designed solution simplifies the complexity of the Public Key Infrastructure (PKI) environment supported by the OneSpan Authentication Server Framework (formerly VACMAN Controller) solution. Integration and implementation is a fast and simple process.
- **Scalable**
  You can add more users, and use the tokens to secure more and other business applications.

Key Features

- **Multi-Factor Authentication**
  This adds additional security measures to the standard username, password logins across the AccessMatrix UAS system that stops unauthorized logins, even the passwords that have been shared among different services.
- **Token Management**
  The administrator can determine and manage the user-to-token relationships.
- **Token Grouping Management**
  The administrator can configure the token batches and groups to the respective users.
- **Token Integration APIs**
  OneSpan token offers APIs integration to the AccessMatrix UAS system.
- **Ready Integration with Leading HSM Vendors**
  Integration is easy to carry out with the existing HSM vendors.
- **Security Auditing and Reporting**
  Audit reports are available in a wide selection of PDF, HTML and XML formats.
- **Scalable and High Availability Architecture**
  It supports applications that can utilize industry-standard protocols and applications along with high-availability support.

Manage OneSpan Token Life Cycle

AccessMatrix supports multiple authentication methods to enable organizations to deploy the appropriate level of authentication technologies to address security requirements. The following describe how you manage the life cycle of OneSpan tokens.

- **Token Initialization**
  OneSpan, the factory initialization, normally carries out the token initialization. No UAS component is involved in the token initialization process. Token initialization has a proper control procedure to manage the token seeds and the encryption keys that will be used by different persons.
    - **Transport Key**
      This is the encryption key generated by OneSpan to protect the DPX file. It can be separated into several key components to be used during the token import process.

- **Digipass Key File (DPX)**

  This file contains token information such as the token seeds and the token serial numbers that can be single-encrypted or double-encrypted. Once the logical token information is set up in the DPX file, it is imported to the server using UAS Token Import Utility.

- **Token Key**

  The token keys are physical tokens that will be grouped into batches for different users.

- **Token Administration**

  The system administrator sets up the token parameters, token groups and batches, and the administrators' rights.

  - **Token Import**

    Token import is a bulk job process which is also an asynchronous process that will run in the background. First, the token seeds are imported into the UAS repository using the Token Import Utility (TIU). Token Import Utility requires two administrators assigned with Token Import right to log in. Once the tokens are imported successfully, they will be in a re-synchronous state.

  - **Token Inventory**

    The assigned token administrator will take charge of the token inventory in the UAS repository.

Token Management

Manage different models of OneSpan tokens by carrying out the following:

- **Token Assignment**

  Assign an active token to a user by carrying out the token assignment in the Admin Console in UAS. The administrator can use either the System Assigned option or the token serial number to assign to the users. The system assigned method tells the system to assign an available token from a selected token group to the user. Use the token serial number to assign a token to the user(s) manually.

  > **Note**: Only the administrator with the assigned token right can assign a token to a user.

- **Token Un-Assignment and Status Change**

  The administrator can unassign the tokens from the users and change the token status in the Admin Console.

> **Note**: You need to change the status of an un-assigned token to "active" before it can be re-assigned.

- **Token Re-Sync**

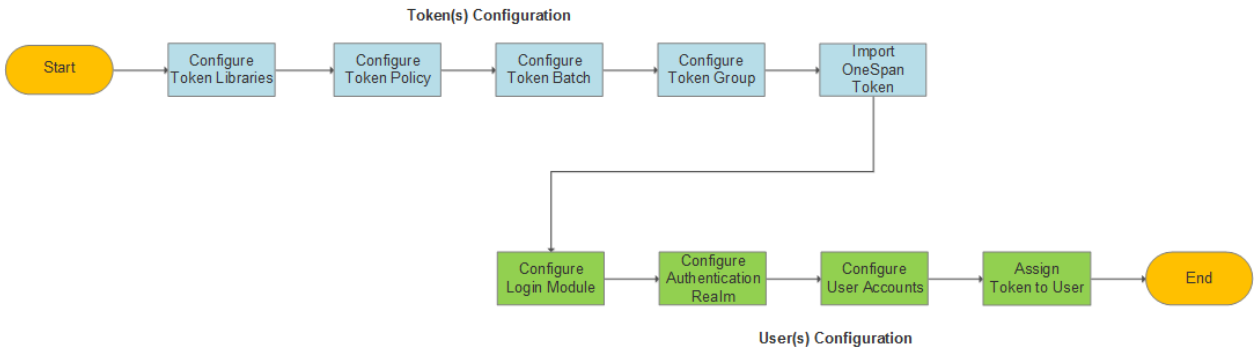
  Token may be out-of-sync with the security server due to the time drift from the GMT. To resolve this problem, the user's token needs to be resynchronized.

- **Replacement over time**

  The administrator can set the life span of a token to be replaced after a specific timeline period on the token status field.

Implementation Checklists

To use OneSpan tokens for multi-factor authentication on AccessMatrix UAS, you must first configure the tokens, the user accounts, and then assign the tokens to the respective users for authentication. See the following flowchart to set up the tokens and the users in AccessMatrix UAS.



> **Notes**:
>
> - You must configure the token policy before configuring the login modules for the users, as the token policy is needed when you configure the login modules.
> - After configuring the token libraries, to carry out the token administration works, you can configure the global settings for the token management, and set up the Token Import Utility (TIU) before you configure the token policy.

Token(s)	Reference
Configure token libraries	Configure OneSpan Token Libraries
Configure token policy	Configure OneSpan Token Policy
Configure token batch	Configure Token Batch

Token(s)	Reference
Configure token group	Configure Token Group
Import OneSpan token	Import OneSpan Token
User(s)	**Reference**
Configure login module	Configure Login Module
Configure Authentication Realm	Configure Authentication Realm
Configure user accounts	Configure User Accounts
Assign token to user	Assign Token to User

Implementation Planning

During implementation, you need to understand the customer's nature of business, studying the customer's requirements and its existing environments to carry out the action plans. The following discuss the security and integration considerations.

Security Consideration

The token policy set by the administrator defines the behaviors of the authentication and authorization for security purposes. The administrator needs to identify and set the token policy in the admin console. The implementation "Vasco Kernel Parameters" contains the required parameters to work with UAS. You have to identify which method of implementation to use. The following show different methods of integration with OneSpan tokens:

- The groups of tokens are assigned to the users manually (OneSpan tokens' serial numbers >> users).
- Generate an authorization code through the UAS system.
- Use API integration.
- Use Digipass online activation Secured Data Exchange protocols.
- Use the encrypted DPX transport keys.

OneSpan token offers strong authentication by using the two protocols: Secure Remote Password (SRP) and Elliptic Curve Diffie-Hellman (ECDH) protocols. In addition, both protocols encrypt the users' information to enhance the security of your remote access.

SRP protocol uses the developer's app for activation and calls OneSpan tokens, whilst the ECDH protocol uses the Digipass for mobile apps (originally from OneSpan) for activation that you do not need to log in and use the authorization code.

Integration Consideration

Before the users can use the tokens, tokens must first be assigned to them. You set the relationships between the tokens and the users first before assigning the tokens to the users. The information (user-to-token relationships) of assignment can be stored either in UAS or in the application database, depending on the tokens used by one application or multiple applications.

- **User-to-token relationships (assignment/unassignment) are maintained by UAS**
  AccessMatrix Universal Authentication Server (UAS) maintains the token serial number and user Id mapping information. You do not keep this information including the token serial number in the application database if:
  - User-to-token serial number relationships are stored in UAS.
  - Multiple applications need to share the tokens, use UAS to maintain the user-to-token relationships.
- **User-to-token relationships are maintained by the application database**
  The application database maintains the token serial number and user Id mapping information. You do not keep this information including the token serial number in AccessMatrix UAS if:
  - User-to-token serial number relationships are stored in the application database.
  - Only one application needs to use the tokens, use the application database to maintain the user-to-token relationships.

3. UAS OneSpan FM with SafeNet ProtectServer HSM

> **Note:** Skip this section if you do not intend to use OneSpan-based services with HSM(s).

This page provides information on the concept and implementation of the AccessMatrix Universal Authentication Server (UAS) with the OneSpan (formerly VASCO) Functionality Module (FM) and SafeNet ProtectServer Hardware Security Module (HSM). The OneSpan Functionality Module (FM) is a designed, security-sensitive program code application which can be uploaded into the HSM and operate as part of the HSM firmware to provide OneSpan services.

The solution provides integration, administration, signing and authentication services by using the OneSpan FM with SafeNet ProtectServer HSMs to protect the cryptographic keys that contain sensitive data against compromises and attacks. See the required components below for the implementation.

- **Hardware**
    - SafeNet ProtectServer Network Hardware Security Modules (HSMs)
    - AccessMatrix UAS server(s)
- **Software**
    - AccessMatrix UAS application
    - SafeNet ProtectServer HSM drivers - Protect Toolkit C version 5.7.0, with the firmware below that is used for communications:
        - SafeNet Network HSM Access Provider 5.7.0
        - SafeNet ProtectToolkit C Runtime 5.7.0
        - SafeNet ProtectToolkit C SDK 5.7.0
    - Vasco Vacman Controller SDK for SafeNet Protect Server HSMs for the platform(s) your company uses (i.e. Windows or Linux).

> **Note:** Always use the latest version of the drivers bundled with the HSM.
> - The Vasco Vacman Controller FM SDK is the OneSpan FM.
> - Supported environment: Oracle JRE/JDK 1.8 (minimum is 1.8.0_181_b13).
> - Supported platforms: Windows, Linux and Solaris. This guide and related guides focus mainly on Windows and Linux platforms.

Implementation Checklist

To use OneSpan FM SDK as the host application, you must first install the SafeNet ProtectServer HSM drivers. All of the drivers and software required to install and configure OneSpan FM SDK is included in the Protect Toolkit C version 5.7.0 with firmware. Set up the environment before installing SafeNet HSMs drivers and OneSpan FM.

Make sure the following configurations are carried out in SafeNet ProtectServer HSM.

- HSM Adapter is initialised.
- OneSpan Token within the slot is initialised.
- OneSpan Functionality Module (FM) is signed and uploaded to SafeNet ProtectServer HSM.
- SafeNet ProtectServer HSM is running properly.

Below is the checklist for implementing OneSpan FM with SafeNet ProtectServer HSM:

Task	Reference	Guide
Install Oracle JDK/JRE	Install Oracle JDK/JRE	*UAS SafeNet ProtectServer HSM Deployment Guide - Deploy SafeNet ProtectServer Network HSM*
Install SafeNet ProtectServer HSM Drivers	Install SafeNet ProtectServer HSM Drivers	
Initialize HSM Adapter	Initialize Adapter	*UAS SafeNet ProtectServer HSM Deployment Guide - Initialize Token and Change Pin (Windows OS)*
Initialize Token Slot	Initialize Token	
Change Token User PIN	Change Token User PIN	
Change Security Officer PIN	Change Security Officer PIN	
Install OneSpan FM	Install OneSpan Functionality Module (FM)	*UAS OneSpan Token Implementation Guide - UAS OneSpan FM with HSM Deployment*
Upload OneSpan FM to SafeNet ProtectServer HSM	Upload OneSpan FM to SafeNet ProtectServer HSM	
Create HSM Keys	Create HSM Keys	*UAS OneSpan Token Implementation Guide - UAS OneSpan FM with HSM Configuration*
Configure Admin Console	Configure Admin Console	
Assign Token to User	Assign Token to User	

Task	Reference	Guide
Test OneSpan FM Token with SafeNet HSM	Test OneSpan Functionality Module token with SafeNet HSM using VACMAN Controller	
Configure SafeNet JCProv Failover	UAS OneSpan FM with SafeNet JCProv Failover	*UAS OneSpan Token Implementation Guide - UAS OneSpan FM with SafeNet JCProv Failover*

Implementation Planning

During implementation, you need to understand the customer's nature of a business, studying the customer's requirements and its existing environments to apply the enterprise security policy and key management. It may be necessary to conduct a comprehensive risk assessment with the customer to reveal the possible points to design key management policies and its procedures. Key management is primarily based on cryptographic techniques such as Diffie-Hellman (DH) key agreement or RSA-based key transport.

The following sections discuss the implementation considerations that should be assessed during planning.

Security Considerations

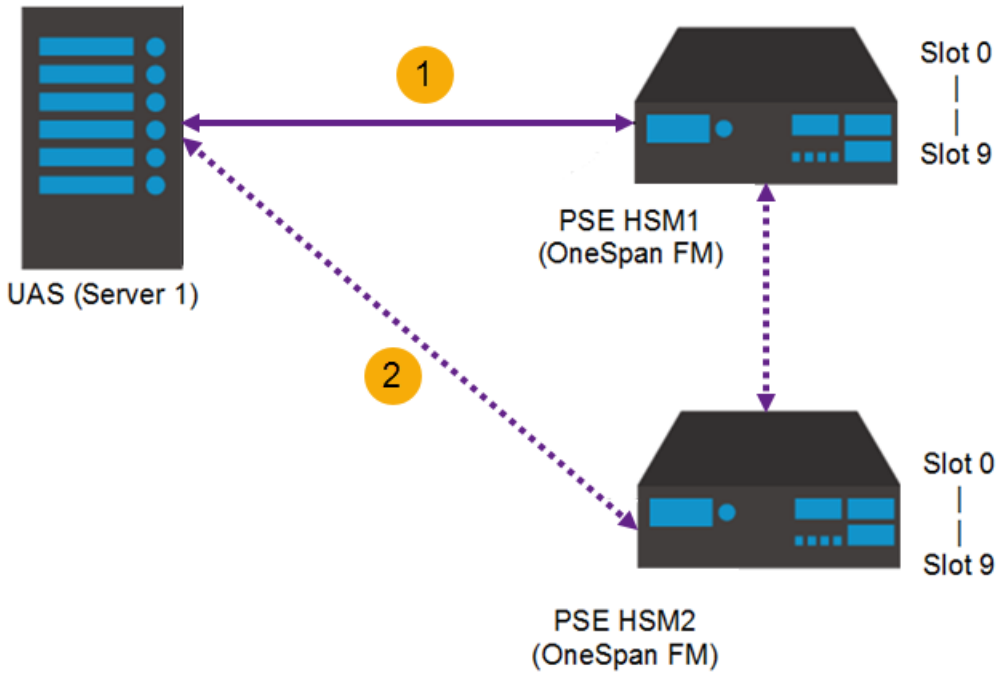
HSMs securely manage and stores digital keys used during transactions, for identification, and applications access. Check the security services your HSM provides when you carry out this implementation.

- Upon successfully carrying out the integration of OneSpan FM with the SafeNet ProtectServer HSM, the data is typically protected based on the application, with **OneSpan FM** handling the data.
- By using the **HSM Database Storage Key,** the data can be of any size and typically protected for any authorized entities on AccessMatrix UAS server(s).

  Note: If you have a second UAS server, the same database can be connected from the first UAS server to the second UAS server.
- When defining the token policies, it is necessary to make them intuitive for the security officer and administrator to carry out their jobs.

- Together with the customer, it is essential to provide security services by defining the **key strengths' standards** and **relationships** to meet the organization's requirements. Refer to the section UAS OneSpan FM with HSM Configuration to specify the relationships, and using which key types and mechanisms, including its class and category when creating the secret keys.

For the architecture design, it is recommended to use a minimum of **two HSM devices with appropriate backups** when integrating with OneSpan FM. Once an HSM is enabled, the operation cannot be undone since it is a one-way, irreversible process. Also, if the first SafeNet ProtectServer HSM fails, AccessMatrix UAS will connect to the second SafeNet ProtectServer HSM.



Integration Considerations

You must install and configure SafeNet ProtectServer HSM device properly based on the *UAS SafeNet ProtectServer HSM Deployment Guide*. All of the drivers and software required to install and configure the OneSpan FM SDK is included in the Protect Toolkit C version 5.7.0 with firmware. Set up the environment before installing SafeNet HSMs drivers and OneSpan FM.

During the integration of OneSpan FM with SafeNet ProtectServer HSM, ensure you follow the below prerequisites:

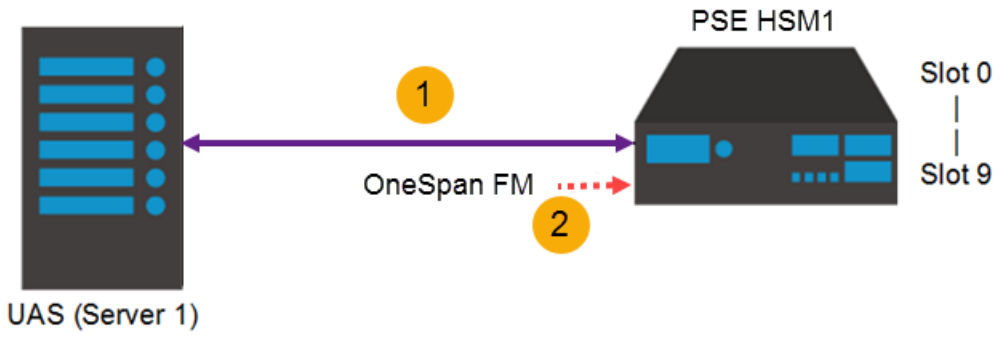
- HSM Adapter is initialized.
- OneSpan Token within the slot is initialized.
- OneSpan Functionality Module (FM) is signed and uploaded to SafeNet ProtectServer HSM.
- SafeNet ProtectServer HSM is running properly.

Performance Considerations

It is necessary to enforce the key management roles to allow a high level of accountability for the performance of key management operations. A well-defined set of users, administrators and security officers ensure that adequate operational controls can be enforced throughout the key management lifecycle. For example, impose multi-person control over critical operations. There must have relevant controls over the policy assets for different types of cryptography and keys used in the data protection applications.

3.1. UAS OneSpan FM with SafeNet ProtectServer HSM Deployment

See the setup diagram below. You deploy the AccessMatrix UAS server, SafeNet ProtectServer HSM, and then upload the OneSpan Functionality Module (FM) to SafeNet PSE HSM.



Deploy AccessMatrix UAS to communicate with SafeNet ProtectServer HSM on Windows

Install AccessMatrix UAS application server and connect it to SafeNet ProtectServer HSM before downloading OneSpan FM to SafeNet ProtectServer HSM.

> **Note**: It is assumed that the SafeNet ProtectServer HSM device is already installed and configured before performing the following steps in this section. Refer to *UAS SafeNet ProtectServer HSM Deployment Guide* for the deployment details.

Before using the VACMAN Controller FM SDK for SafeNet PSE HSM, you need to upload a VACMAN Controller HSM module to the SafeNet PSE HSM. This section shows you how to install and register the OneSpan FM in the AccessMatrix UAS server, and then upload it to SafeNet ProtectServer HSM.

Install OneSpan Functionality Module (FM)

Windows	Install OneSpan SDK version 4.0.1 on the AccessMatrix UAS server. The OneSpan Functionality Module (FM) is found in the FM subfolder, *<UAS_DIR>\OneSpan\Authentication Suite Server SDK 64 bit Thales ProtectServer HSM 4.0.1\Hsm\k7-ppc\signed*. 1. Install the OneSpan Authentication Server Framework SDK for SafeNet HSM 4.0.1.

<table>
<tr><td></td><td>
2. Copy the library from <InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit Thales ProtectServer HSM 4.0.1\Bin\aal2sdk.dll to one of the UAS server installation folders as listed below:
a. <UAS_DIR>\JRE\bin\ (for embedded Tomcat JRE dependency)
b. <UAS_DIR>\Java\jdk1.8.0_181_b13\jre\bin\ (for Windows Java dependency)
3. Copy the library from <InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit Thales ProtectServer HSM 4.0.1\Wrappers\Java\aal2wrap.jar to the UAS server installation folder,<UAS_DIR>\webapps\am5\WEB-INF\lib\.
</td></tr>
<tr><td>Linux</td><td>
Install OneSpan SDK version 4.0.1 Linux version on the AccessMatrix UAS server. The OneSpan Functionality Module (FM) is found in the FM subfolder, <UAS_DIR> /opt/vasco/Authentication_Server_Framework-HSM-4.0.1/.

Install by using the command <code>rpm -ivh <vacmansdk_package>.rpm</code>.
Example: <code>rpm -ivh aal2sdk_SAFENET-4.0.1-0.x86_64.rpm</code>

Perform the following steps after installation:
1. Run the <code>vi /.bash_profile</code> command.
2. Export the SafeNet PTKC bin and library path by adding the following lines:
<pre>PATH=$PATH:\opt\safenet\protecttoolkit5\ptk\bin export PATH

LD_LIBRARY_PATH=$LD_LIBRARY_PATH:\opt\safenet\protecttoolkit5\ptk\lib export LD_LIBRARY_PATH</pre>
3. Export the SafeNet NETCLIENT server list by adding the following line:
<code>export ET_HSM_NETCLIENT_SERVERLIST=<HSM_IP>:<HSM_PORT></code>
Note: Substitute <code><HSM_IP></code> and <code><HSM_PORT></code> with the actual HSM IP address and port.
4. Save and close the file.
5. Run the <code>hsmstate</code> command to verify whether the HSM Access Provider is installed successfully. The response should be:
<code>HSM device 0: HSM in NORMAL MODE. RESPONDING. Usage Level-0%</code>

For the AccessMatrix UAS server to talk to the HSM with the OneSpan FM installed, the administrator then has to perform the following steps:

1. Copy the VACMAN Controller library aal2wrap.jar from /opt/vasco/Authentication_Server_Framework-HSM-4.0.1/wrapper/java/ to the AccessMatrix UAS server library directory, <UAS_DIR>/tomcat/webapps/am5/WEB-INF/lib/.
2. Copy this VACMAN Controller file libaal2sdk.so from /opt/vasco/Authentication_Server_Framework-HSM-4.0.1/lib/ to the AccessMatrix UAS server binary directory, <UAS_DIR>/tomcat/bin/.
3. Restart AccessMatrix UAS server.
</td></tr>
</table>

Upload OneSpan FM to SafeNet ProtectServer HSM

A signed OneSpan Functionality Module and its signing certificate must be uploaded to the HSM. Both files can be located at *<InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit Thales ProtectServer HSM 4.0.1\Hsm\k7-ppc\signed*.

Perform the following steps:

1. Import the provided signing certificate into the HSM by entering the command `ctcert i -f <vasco_cert_filename> -s<vasco_slot_number> -l<key_name>`.

   For example: `ctcert i -f vascosigningcert_20470309.crt -s1 -lvascosigningkey`

   > **Important Note:** You should use the HSM's Admin Slot while performing the steps detailed in this section. In the above example, it is assumed that slot 1 (i.e. `-s1`) is the HSM Admin Slot; if the HSM Admin Slot is a different slot, use that slot instead.

2. Make the certificate a trusted certificate by entering the command `ctcert t -s<vasco_slot_number> -l<key_name>`.

   For example: `ctcert t -s1 -lvascosigningkey`

3. Upload the signed FM to the HSM by entering the command `ctconf -j <signed_vasco_fm_filename> -b <key_name>` .

   For example: `ctconf -j aal2sdk.signed -b vascosigningkey`

4. After uploading and restarting the FM, verify that the FM has been successfully loaded by entering the command `ctconf -s`.

   You should see the an output similar to the example below:

```
ProtectToolkit C Configuration Utility 7.0.0
Copyright (c) Safenet, Inc. 2009-2020

Current Functionality Module Configuration:

Label        : VC 3.22
FM ID        : 100
Version      : 0.00
Manufacturer : OneSpan
Build Time   : *** No Date/Time ***
Fingerprint  : A17AA9E2-D02CC29A-EA430BF4-FD765240
ROM size     : 377491
RAM size     : 0
Status       : Enabled
```

> **Notes**:

- If any issues are encountered in the FM uploading process, first run the `hsmstate` command to verify whether the HSM Access Provider has been successfully installed.
  The response should be:

```
HSM device 0: HSM in NORMAL MODE. RESPONDING. Usage Level-0%
```

- Refer to B. Utility Reference for more information.

3.2. UAS OneSpan FM with SafeNet ProtectServer HSM Configuration

This section details the configuration steps relating to the implementation of the UAS OneSpan FM with HSMs.

Setup Steps

This section defines the steps relating to the implementation of the UAS OneSpan FM with the SafeNet ProtectServer HSMs. Before you start the implementation, it is assumed that you have completed the planning and implementation checklists, and you have understood the concept of the UAS OneSpan FM with HSM solution.

- **Create HSM Keys**
  A hardware security module (HSM) is a physical computing device that safeguards and manages digital keys for strong authentication and cryptoprocessing. Refer to Create HSM Keys to know the required keys for the UAS OneSpan FM solution.
- **Configure Admin Console**
  To use the OneSpan FM Token, the token policy, token group, and token batch must be configured in the AccessMatrix UAS server. The OneSpan FM Token must also be imported via the Token Import Utility. Refer to Configure Admin Console for details.
- **Assign Token to User**
  Assign OneSpan FM tokens to users for authentication with tighter security via the SafeNet ProtectServer HSM. Refer to Assign Token to User for details.
- **Test OneSpan FM Token**
  After integrating the OneSpan FM with the SafeNet ProtectServer HSM, test the system to ensure that the integration was successful.
  Refer to Test OneSpan Functionality Module Token with SafeNet HSM using VACMAN Controller for details.

Create HSM Keys

SafeNet ProtectServer HSMs provide high security by storing the cryptographic keys in the hardware. Generate the crypto keys using SafeNet Key Management Utility (KMU) tool.

Note: The Key Management Utility (KMU) tool is included in the Safenet HSM SDK (ProtectToolkit C). It provides functions that allow management of keys using a Public Key Cryptography Standard # 11 (PKCS#11) sub-system.

You can carry out the following key management operations using KMU.

- Key generation
- Key import
- Key export
- Key wrapping
- Key unwrapping
- Set and/or modify key properties

See below a list of the expected key attribute values of each type of secret keys. For different customers, there will have different expectations for their secret keys. The list below is customizable according to the requirements of different customers.

Key Name	HSM Database Storage Key	HSM Transport Master Key	Key Encryption Key (KEK)	Functional Module (FM) Signing Key (Private)	Functional Module (FM) Signing Key (Public)
Key Label	vascoStorageKey	vascoTransportKey	KEK	vascoSigningKey	vascoSigningKey
Key Type	Triple DES / AES	Triple DES / AES	Triple DES / AES	RSA (Private)	RSA (Public)
Key Size	192 / 256	192 / 256	192 / 256	2048	2048
Persistent	Y	Y	Y	Y	Y
Private				Y	
Sensitive	Y	Y	Y	Y	
Modifiable				Y	
Wrap	Y	Y	Y		Y

Key Name	HSM Database Storage Key	HSM Transport Master Key	Key Encryption Key (KEK)	Functional Module (FM) Signing Key (Private)	Functional Module (FM) Signing Key (Public)
Unwrap	Y	Y	Y	Y	
Extractable	Y			Y	Y
Export					
Exportable	Y	Y	Y	Y	
Derive					Y
Encrypt		Y	Y		Y
Decrypt			Y	Y	
Sign				Y	
Verify					Y

Notes:

- Triple-length/DES mechanism or key type is highly recommended for the HSM storage key, HSM transport key and key encryption key (KEK).
- To allow backup for the key label "vascoStorageKey", enable the **Exportable** setting.
- To allow backup for the key label "vascoTrasportKey", enable the **Exportable** setting.
- To allow backup for the key label "KEK", enable the **Exportable** setting.
- Enable **Exportable** and disable **Extractable** to the key label "vascoTransportKey" for security reasons.

Meaning of Key Attributes

Attribute Name	Description
Persistent	Indicates whether a key object has been created for all the sessions. If you disable "X", the key is only visible for the current session and will be destroyed when the session ends.
Private	Indicates whether users need to authenticate to the key's HSM token before they can access the key object. Defines whether the user PIN protects the object. A private object is only accessible to an application that has supplied the user PIN.
Sensitive	Indicates whether the key object can be extracted from the Hardware Security Module (HSM) in clear. The key object includes the values of all key attributes. In other words, if a key is **Sensitive**, the key's value will not be revealed in plain text/clear. Once a key becomes **Sensitive**, it cannot be modified to be non-sensitive.
Modifiable	Indicates whether a key object can be changed after creation. Changing the key object involves changing the object's attributes.
Wrap	Indicates whether the key can encrypt other keys that are in the HSM.
Unwrap	Indicates whether the key can decrypt encrypted key material that is in the HSM.
Extractable	Indicates whether the key can be extracted from the HSM in encrypted form. Any user of the HSM can control the key encryption key. It is recommended to use the **Exportable** rather than the **Extractable** property.
Export	This attribute is similar to the **Wrap** attribute in that it specifies that the key may be used to encrypt a second key so that it may be extracted from the HSM in an encrypted form. Unlike the **Wrap** attribute, however, only the security officer may specify this attribute.
Exportable	Indicates whether the key can be exported from the HSM in encrypted form. However, the key encryption key needs to be controlled by a security officer, not by a standard user.
Import	This attribute is similar to the **Unwrap** attribute. It is used to determine if a given key can be used to unwrap encrypted key material.
Derive	Indicates whether other keys can be derived from the key.
Encrypt	Indicates whether the key can be used to encrypt data.
Decrypt	Indicates whether the key can be used to decrypt data.
Sign	Indicates whether the key can be used to generate digital signatures or message authentication codes (MACs).

Attribute Name	Description
Verify	Indicates whether the key can be used to verify digital signatures or message authentication codes (MACs).

The following keys must be generated for the OneSpan FM with HSM solution:

- **HSM Database Storage Key**

  The storage key is used to encrypt the token seeds in the database.
- **HSM Transport Master Key**

  The transport key is used to protect token seeds in transit.
- **Key Encryption Key (KEK)**

  The key encryption key is used to wrap/unwrap the encrypted transport key.

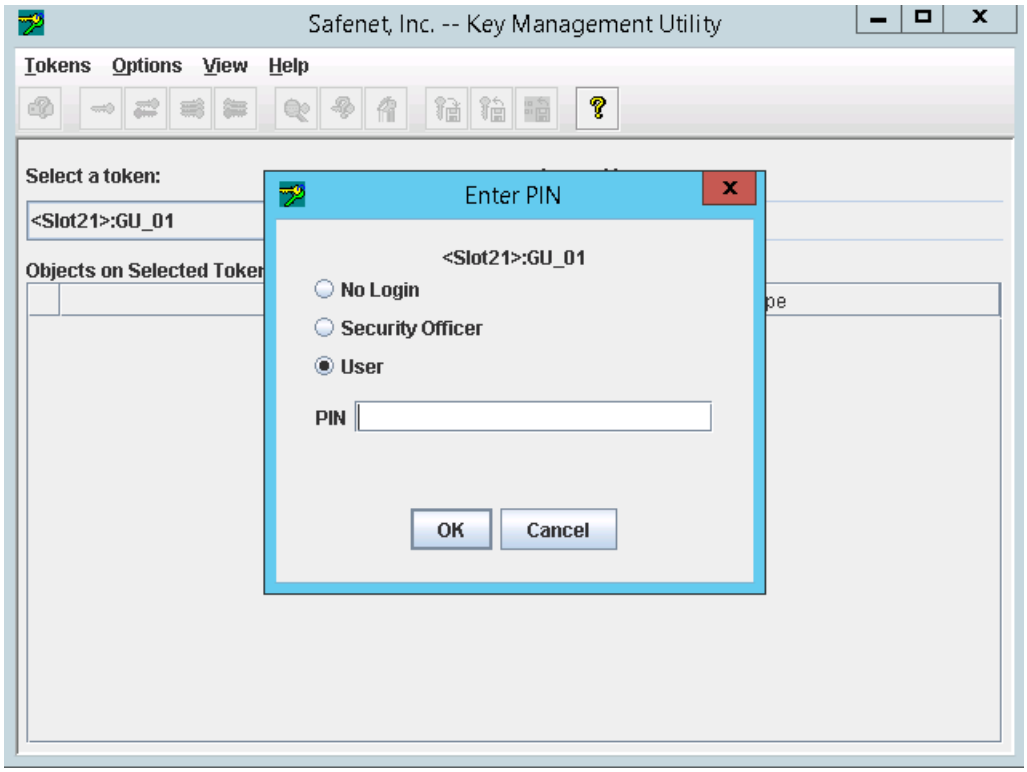
Generate HSM Database Storage Key

This key is used to encrypt the token seeds in the database. Before generating the HSM Data Storage key, ensure the following prerequisites have been met:

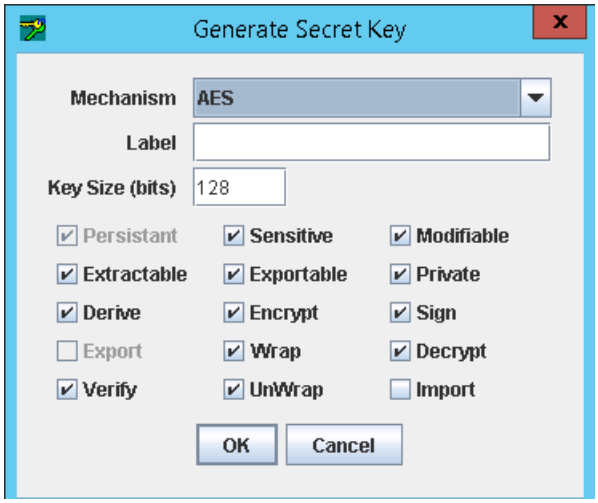
- New slot on the HSM has been created
- A new token has been created and initialized

To generate the HSM Database Storage Key, perform the steps below:

1. Open the SafeNet Key Management Utility (KMU) and select the token that needs to be created with a secret key. The **Enter PIN** dialog box is displayed.

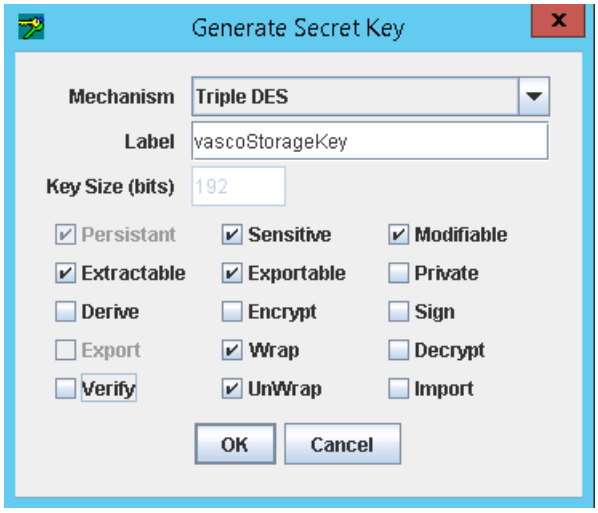


2. Select the **User** radio button to log as a user, and enter the token PIN. Click **OK**.
3. From the menu, select **Options > Create > Secret Key** or use the **Secret Key** icon. The **Generate Secret Key** dialog box is displayed.

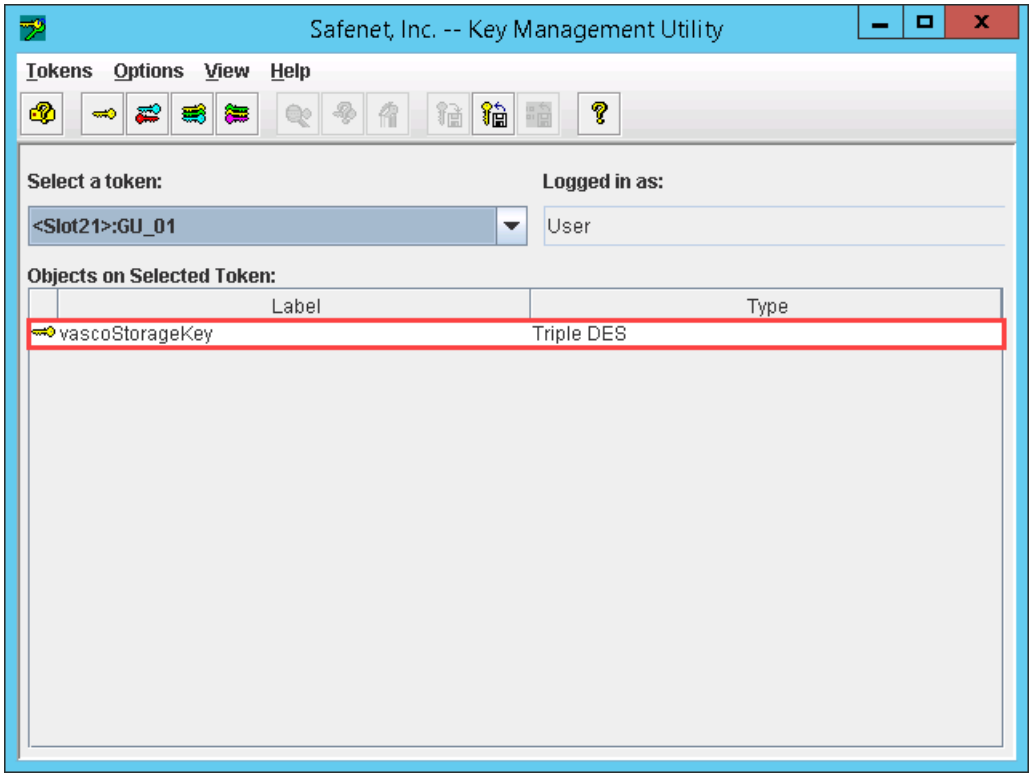


4. Select the type of mechanism. Create the HSM Database Storage Key using "Triple-DES" (with a key length of 192).
5. Enter "vascoStorageKey" as the key's label and select the attributes according to the Key Attributes table in section 4 for the HSM Database Storage key. To allow backup, click the

Exportable checkmark **ON**.



6. Click **Ok**. The vascoStorageKey is generated.



Set SafeNet Security Mode for Multiple Custodians Key

If you are exporting a key using the standard multiple custodians method, set the **Weak Mechanisms** flag to **ON** to generate key components of a transport master key or a key encryption key (KEK). The multiple custodians method is where a key is split into shares and distributed among multiple custodians.

Note: Refer to the SafeNet PDF document, *PTK-C_Administration_Guide* for more information about the security flag options.

1. Open gCTAdmin utility from the SafeNet program folder. (e.g. *C:\Program Files\SafeNet\Protect Toolkit 5\Protect Toolkit C RT\gCTAdmin.bat*).
2. Enter the User PIN and press **Enter** to log in.
3. Select **Edit > Security Mode** on the main menu. The **Manage Tokens** dialog box is displayed.
4. Select the **Custom** radio button to display the **Custom Security Flags** dialog box.
5. Click the **Weak PKCS#11 Mechanisms** checkbox **ON** and click **OK** to close the dialog box.
6. Click **OK** in the **Modify Security Mode** dialog box. A restart alert message is displayed.
7. Click **OK** to restart CtAdmin. You can now proceed to create the key components. The key components can be created by using either the **Generate Key Components** or **Enter Key from Components** function.

Important Note: This security flag is not necessary when using the Single Custodian method where the key is used to encrypt a key or a set of keys to be exported. Also, by setting this flag **ON**, it compromises security. After generating the key components, repeat the steps above and then turn **OFF** the **Weak PKCS#11 Mechanisms** flag.

Generate HSM Transport Master Key

It is necessary to generate a transport key to be available for transfer them from the generation site to the location where it will be used. Transport key should be achieved securely by SafeNet Key management techniques. Use SafeNet Key Management Utility (KMU) to generate the transport master key securely.

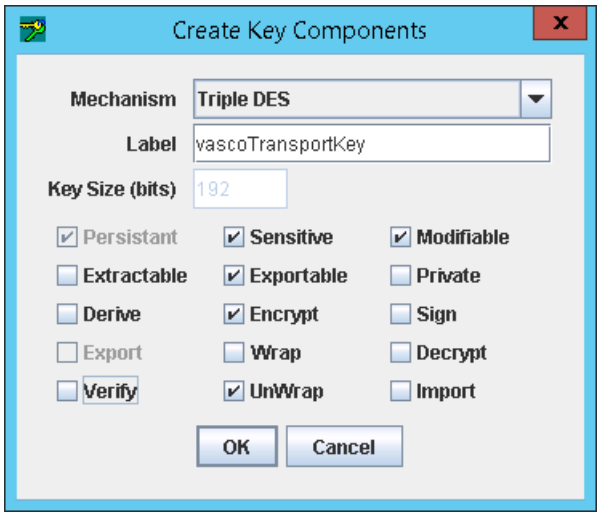
Note: Set SafeNet Security Mode for Multiple Custodians key before proceeding.

If no component key is provided, follow the steps below to generate the Transport Master keys.

1. Open SafeNet Key Management Utility (KMU) and select the token that needs to be created with a new key component. The **Enter PIN** dialog box is displayed.
2. Select the radio button to log in by user, and enter the token pin. Click **OK**.

3. Select **Options > Create > Generate Key Components** or the **Generate Key Components** icon.

   The **Create Key Components** dialog box is displayed.



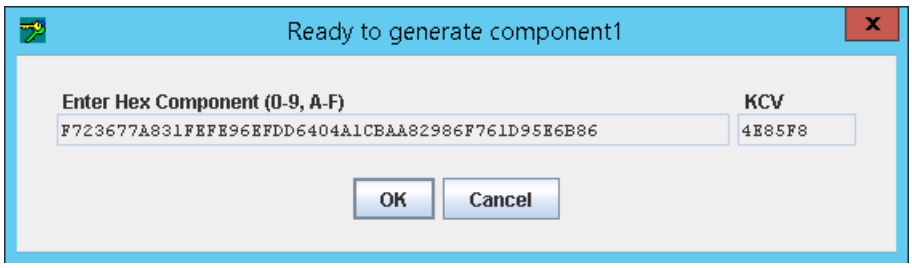
4. Enter the key's label and select the mechanism and the key attributes according to the Key Attributes table in section 4. Click **OK**.

> **Note**: For security purpose, click the checkmark **OFF** for **Extractable** and the checkmark **ON** for **Exportable** to prevent the HSM Transport key from being extracted and easily exportable outside the HSM to compromise its secret key.

   Enter the number of components needed to be created for the key.

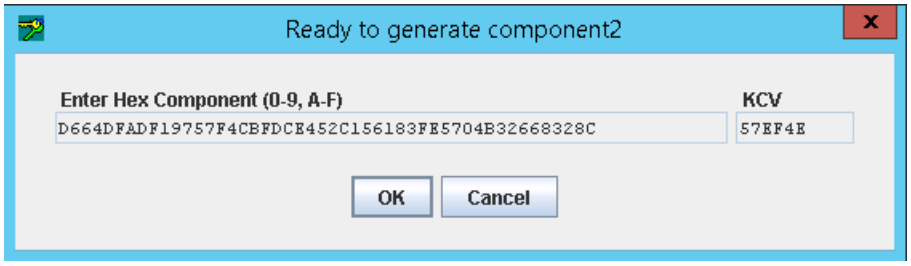
> **Note**: At least 2 components must be created. Type 2 for the number of components. Click **OK**.

5. The message box, "Ready to generate component1" is displayed. Click **OK**. The system generates random numbers for key components based on the mechanism chosen.



6. Click **OK**. The information message box displays and prompts "Ready to generate component 2". Click **OK** to accept and generate key component 2. The system generates random numbers for

key components based on the mechanism chosen.

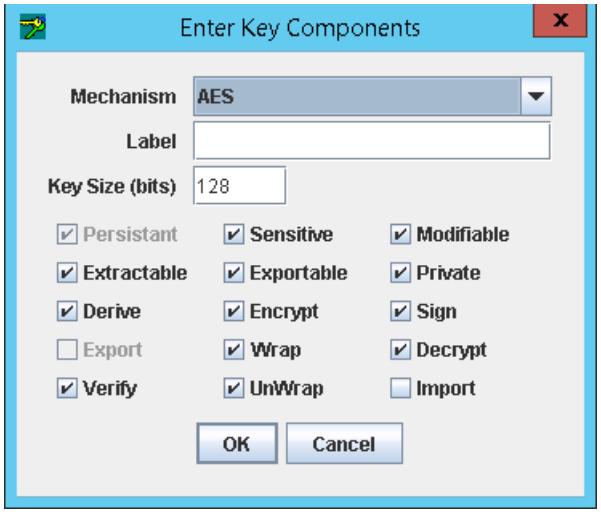


7. Click **OK**. A transport master key is generated.

If there are component keys provided, follow the steps below to generate the Transport Master key.

Note: Ensure that the component keys are ready for import.

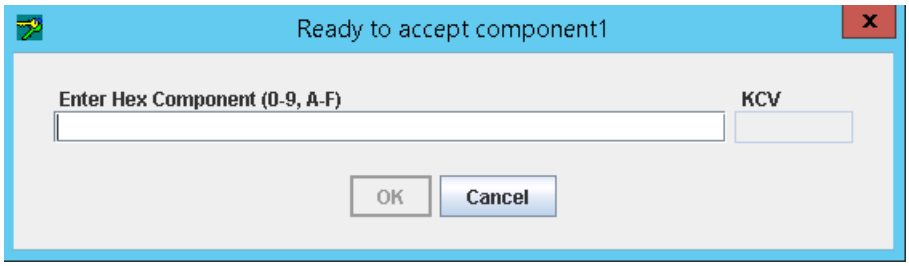
1. Open SafeNet Key Management Utility (KMU) and select the token that needs to be created with a new key component. The **Enter PIN** dialog box is displayed.
2. Select the radio button to log in by the user, and enter the token pin. Click **OK**.
3. Select **Options > Create > Enter Key from Components** or press the **Enter Key from Components** icon. The **Enter Key Components** dialog box is displayed.



4. Enter the key's label and select the mechanism and the key attributes according to the Key Attributes table in section 4. Click **OK**.
5. Enter the number of components needed to be created for the key.

Note: At least 2 components must be created. Type "2" for the number of components. Click **OK**.

6. The message box, "Ready to generate component1" is displayed. Click **OK**. The system prompts for the key components. Enter the component value and the KCV value.



7. Click **OK**.
   The message box displays and prompts "Ready to generate component 2".
8. Click **OK**. The system prompts for the key components.
9. Enter the component value and the KCV value, then click **OK**. A transport master key is generated.

Generate HSM Key Encryption Key (KEK)

Use KMU to generate the key-encryption keys for high-security purpose in order to secure a large database system or a storage network using the concept of key wrapping. The purpose of this key encryption key (KEK) is to export the HSM DPX transport key. KEK is a special key that is created with the **Wrap** attribute that is usually created as split custodian keys because of their high-security nature. Same as the procedural steps as listed on the section Generate HSM Transport Master Key. Enter the key's label, select the mechanism and the key attributes according to the Key Attributes table on section 4 for the key encryption keys.

Notes:

- As an HSM KEK is also created using the multiple custodians method, turn **ON** the **Weak Mechanisms** flag to generate the key components before creation. Refer to Set SafeNet Security Mode for Multiple Custodians Key for more information.
- Specify the number of KEK components when prompted for the "Number of components to create".

Configure Admin Console

Validate Token Vendor

To use OneSpan FM token, AccessMatrix UAS server requires its server license be enabled with the **VASCO_FM_SAFENET** token vendor. This vendor is required during OneSpan (VASCO) FM token import.

1. Navigate to **Administration Dashboard > System > Server & Cache > Server**.
2. Select the **Server Id** of the server.
3. Select the **Licensed Features** tab, check on the **UAS FEATURE** under the **License Module** column. See if the value **VASCO_FM_SAFENET** is listed in the token vendors' list. If it exists, then UAS feature is granted with OneSpan (VASCO) FM vendor's license.

Create Token Policy

Configure SafeNet ProtectServer HSM characteristics by creating a new token policy via the AccessMatrix Admin Console.

1. Navigate to **Administration Dashboard > Security Policy > Token > Token Policy**.
2. Click the **Create** button and then select "VASCO Kernel Parameters (SafeNet HSM FM Enabled)" token policy implementation.
3. Select **Next**.
4. Enter a **Token Policy Id** and a **Token Policy Name**.
5. Specify the respective HSM eracom slot Id under the ***hsm.eracom.SlotId** attribute to be integrated with OneSpan FM.
6. Refer to the following attributes in the table below to specify the values in the required fields.
7. Click **Save**. A new token policy is created.

Attribute	Description
*Token Policy Id	Identification assigned to the token policy.
*Token Policy Name	The name of the token policy.
Description	Brief description of the token policy.
Version	The token policy revision.
Implementation	Implementation type.
Encryption key	Encryption key for this token policy.
Attributes Tab	
VASCO Specific	

Attribute	Description
Storage Derive Key 1	Derivation key part 1 used to make data software encryption unique for a host.
Storage Derive Key 2	Derivation key part 2 used to make data software encryption unique for a host.
Storage Derive Key 3	Derivation key part 3 used to make data software encryption unique for a host.
Storage Derive Key 4	Derivation key part 4 used to make data software encryption unique for a host.
Check Challenge	Verify if the challenge has been corrupted before validation. • "0"=No password checking • "1"=Check parameter then verify with the DPData Challenge • "2"=Always use the DPData Challenge to validate responses • "3"=Avoid Challenge-Response Replay Attack by allowing only one Challenge-Response authentication per timestep • "4"=Avoid Challenge-Response Replay Attack by rejecting the second response if responses from 2 consecutive authentication requests are equal and in the same timestep Default: 0
Check Inactive Days	Refers to the acceptable number of days of user/token inactivity. Past this number, return code 205 will be generated and the Digipass will have to be reset. "0" means this feature is disabled. Default: 0
Derive Vector	Vector used to make data encryption unique for a host. Default: 0
Diag Level	Level of diagnostic messages generated by functions, which goes to standard output. Default: 0
Event Window	Event Window size, expressed in a number of iterations. This represents the acceptable event counter difference between DIGIPASS and host. This parameter applies only for event-based operating modes. Default: 100

Attribute	Description
GMTAdjust	GMT Time adjustment to perform in case the C language gmtime function does not give an accurate value. Default: 0
HSM Slot Id	HSM Slot Id used to store Storage Key and Transport Key. It is applicable only for HSM integration. Default: 0
IThreshold	Number of successive Identification errors allowed. When the specified number is reached, return code 202 is sent to the caller. The token will then need to be resynchronized. "0" means this feature is disabled. Default: 3
ITime Window	The number of timesteps that determines the acceptable time difference between a Digipass and the host system for Identification function. This difference is adjusted to the last known shift for each token. The maximum drift correction acceptance is 1 second per 6 hour period. The time step is determined by the Digipass internal programming options. Refer to the concepts pages to have more information concerning Time Management in the VACMAN Controller. Value should not be less than 2. Default: 10
ITime Window 1	Additional ITime Window values if the client wanted to use different time window during run time. User can specify the ITime Window to follow during API call.
ITime Window 2	
ITime Window 3	
Online SG	Level of Online Signature. • "0"=Disabled feature • "1"=Several Signatures are allowed in the same TimeStep (except identical successive ones) • "2"=Only one signature per timestep allowed Default: 2
SThreshold	Number of successive Signature errors allowed. When the specified number is reached, return code 203 is sent to the caller. The token will then need to be resynchronized. "0" means this feature is disabled. Default: 3
STimeWindow	The number of timesteps that determines the acceptable time difference between a Digipass token and the host system for Signature function. This difference is adjusted

Attribute	Description
	to the last known shift for each token. The time step is determined by the Digipass internal programming options. Value should not be less than 2. Default: 10
STimeWindow 1	Additional STime Window values if client wanted to use different time window during run time. User can specify the STime Window to follow during API call.
STimeWindow 2	
STimeWindow 3	
Storage Key Id	Key Id used to read (Decrypt) Digipass Blob from a database. Default: 0 **Note:** This attribute is currently unused for Safenet PSE HSMs.
Synchronize Window	For InitialTimeWindow Configuration. This parameter sets the initial derivation between a DIGIPASS time and the VACMAN Controller GMT Time. **Note:** This value is expressed in hours. InitTimeWindow=(max(STimeWindow or ITimeWindow), SyncWindow) Default: 2
Synchronize Window 1	Additional Synchronize Window values if the client wanted to use different time window during run time. User can specify the Synchronize Window to follow during API call.
Synchronize Window 2	
Synchronize Window 3	
TransportKeyId	Key Id used to write (Encrypt) Digipass Blob to the database. Default: 2 **Note:** This attribute is currently unused for Safenet PSE HSMs.
Preferred application ID for OTP (RO) if more than one blob	Specify Preferred Application ID for RO blob to be used for verification if there is more than one blob.
Secure Channel Application ID(s)	Specify the Application ID(s) for Secure Channel application that matches the Application ID(s) defined on OneSpan's (VASCO) parameter sheet.
Secure Channel Message Validity Time	This value indicates the Secure Channel (SC) Message validity time in seconds. During Secure Channel verification, this validity time is used to check whether the

Attribute	Description
	generated Secure Channel message has expired. The default value means this feature is disabled. Default: 0
VASCO Double DPX	Enable OneSpan (VASCO) Double Encrypted DPX import. Default: false
Secure Channel Message Show Warning	If set to "true", the client device or application must display a predefined warning message to the user. Default: true
hsm.eracom.UseSingleSession	HSM Connection to use single session only.
hsm.eracom.StorageKeyIV	HSM storage key initial vector.
*hsm.eracom.StorageKeyName	Refers to the OneSpan (VASCO) storage key name. **Note:** Specify "vascoStorageKey" as the key name, as this is hardcoded in AccessMatrix UAS.
hsm.eracom.LoginPin	HSM slot login pin/password.
hsm.SlotId	Refers to the token slot specified within a single ProtectServer HSM. If multiple HSMs are used, the slot index to has to be the same across all HSMs. **Note:** This attribute is currently unused for Safenet PSE HSMs.
Secure Channel Message Ask PIN	If set to "true", the client device or application must request PIN verification before generating the MAC. Default: false
hsm.DPXTransportKEK	OneSpan (VASCO) DPX Transport Key Encryption Key (KEK).
Secure Channel Message Ask Approval	If set to "true", the client device or application must request the user to confirm the transaction data before the MAC is generated. Default: true
Secure Channel Message Show Data	If set to "true", the client device or application must display the content of the transaction message. Default: true
*hsm.eracom.SlotId	Refers to the token slot specified in ProtectServer HSM. For failover support, refer to SafeNet Jcprov Failover. **Note:** For Safenet PSE HSMs, this attribute is used during the HSM connection and initialization stages. It may be different from the slot used by the FM.

Attribute	Description
hsm.LoginPin	The HSM slot login pin/password **Note:** This attribute is currently unused for Safenet PSE HSMs.
hsm.eracom.LoginRequired	Indicate whether HSM login is required. **Note:** This attribute is currently unused for Safenet PSE HSMs.
allowDecryptSecureChannelMessageAPI	If set to "true", the REST API for decrypting secure channel message will be allowed. Default: false
Secure Channel Message Show MAC	If set to "true", the client device or application must present the calculated MAC to the user. Default: true
Digipass for APPS/Mobile Specific Tab	
Digipass for Mobile Specific	
Server Activation Challenge Response	Enable challenge response verification for activation process if set to "true". Default: false
Score Based Response	Enables Score Based Response (contextual) verification if set to "true". This feature will perform verification on OTP response, which contains User, Context and Platform scoring generated by user's OneSpan (VASCO) token. Default: false
Generate & Send Mobile Activation QRCode After Assignment	This feature is strictly for Mobile Tokens only. If enabled, the server will generate Mobile Activation QRCode upon successful token assignment. This Activation QRCode will be sent to the user through the event notification policy. Select the corresponding event notification policy in the **Event Notification Policy for Activation QRCode** attribute. Default: false
Include Activation Password in QRCode	Set to "true" to embed Activation Password into the Online Activation QRCode. Otherwise, configure event notification policy to deliver the Activation Password. Default: true
Include Authorization Code in QRCode	Set to "true" to embed the Authorization Code into the Online Activation QRCode. Otherwise, configure event notification policy to deliver the Activation Password. Default: true

Attribute	Description
Event Notification Policy for Mobile Activation QRCode	Specify the event notification policy to be used for sending the Mobile Activation QRCode.
Allow Activation Checking	If set to "true", special validation will be performed during Digipass Mobile token activation to check if the token's **Allow Activation/Reactivation** flag is set to **YES**. If token's **Allow Activation/Reactivation** flag is set to **NO**, the token activation will be denied. Set to "false" to disable this validation. Default: false
Allow API For Decrypting Secure Channel Message	If set to "true", the REST API '/rest/VASCO/multidev/message/secureChannel/decrypt' will be allowed. Default: false
Digipass Online Activation Secured Data Exchange Protocol	
Secured Data Exchange Protocol	Specify the type of protocol to be used for Secured Data Exchange during Digipass for APPS/Mobile online activation. • "SRP" - Secure Remote Password protocol • "ECDH" - Elliptic curve Diffie-Hellman protocol Default: ECDH
SRP Activation Specific	
User Password Length	SRP's user password length requirement. Default: 8
User Password Character Sets	SRP's user password character sets requirement. Default: Numeric
User Password Letter Case	SRP's user password letter case requirement. Default: Mixed Case Alphanumeric
Checksum on User Password	If set to "true", OneSpan (VASCO) SDK calculates the checksum of the user's password. Default: true
User Password Validity Time (Seconds)	SRP's user password validity time. Default: 86400
EDCH Activation Specific	
Activation Password Length	This value indicates the activation password validity time in seconds. During password verification, this validity time is used to check whether the generated password has

Attribute	Description
	expired. Default: 86400
Activation Password Character Set	EDCH's activation password character requirement.
Activation Password Validity Time (Seconds)	EDCH's activation password validity time.
Authorization Code	
Authorization Code Length	Authorization code's length requirement. Default: 6
Authorization Code Character Set	Authorization code's character set requirement. Default: Numeric
Authorization Code Validity Time (Seconds)	This value indicates the authorization code validity time in seconds. During authorization code verification, this validity time is used to check whether the generated authorization code has expired. Default: 86400
Authorization Code Retry Threshold	Maximum failed attempts for authorization code verification during activation. The authorization code will be invalidated upon the threshold reached. Set to "0" to disable this feature. Default: 3
CrontoSign Configuration	
Use Dynamic CrontoSign	Setting to "true" will enforce the use of dynamic size CrontoSign image generation to support up to 1070 characters for Secure Channel transaction signing data. If disabled, standard size CrontoSign (support up to 200 characters) will be used instead. Default: false

Create Token Group

Create a token group for OneSpan (VASCO) FM token:

1. Navigate to **Operation Dashboard > Token > Token Group**.
2. Click **Create** to create a new token group.
3. Specify the **Token Group Id**, **Token Group Name** and **Token Vendor**.

> **Note**: Make sure that **VASCO_FM_ERACOM** is selected as the **Token Vendor**.

4. Click **Save**.

Create Token Batch

Use a token policy with the "VASCO Kernel Parameter (SafeNet HSM FM Enabled)" implementation for the token batch. To create this token policy, refer to Create Token Policy section.

To create a token batch:

1. Navigate to view Server details by selecting **Operation Dashboard > Token > Token Batch**.
2. Click **Create** button to create a token batch.
3. Specify the **Token Batch Id**, **Token Batch Name** and **Token Policy**.

> **Note**: The token policy refers to the VASCO Kernel Parameter (SafeNet HSM FM Enabled) policy created earlier.

4. Click **Save**.

Use Token Import Utility (TIU) to import OneSpan FM Token

> **Note**: If the token batch and token group are created in TIU, ensure that the following criteria are fulfilled:
> - The AccessMatrix server license supports the **VASCO_FM_ERACOM** token vendor.
> - Use the token policy created in Create Token Policy section for the token batch.
> - To create a token batch, refer to Create Token Batch.
> - Use "VASCO_FM_ERACOM" as the token vendor for the token group. To create a token group, refer to Create Token Group.

Import Single/Double Encryption DPX

Use the same steps below to import single or double encryption DPX file. The only difference between single and double encryption DPX is that the double encryption DPX requires two transport keys.

Double encryption DPX is encrypted by software transport key and HSM transport key. The HSM transport key must be first created in the HSM. The HSM transport key label is specified within the DPX file. You may open the DPX file with a text editor to confirm the HSM transport key label.

1. Launch Admin Console and click **Cancel** to remove the current login screen.
2. Navigate to TIU by selecting **Token > Token Import Utility**.
3. Enter **Login Id** & **Password for Login** (User 1), e.g. "tokenadmin" / "password". Then, click **OK**.
4. Enter **Login Id** & **Password for Login** (User 2), e.g. "tokenadmin1" / "password". Then, click **OK**. The system shows the **Token Import Utility (TIU)** page.
5. Click on **Import Token**.
6. Upload the DPX file and specify the token batch with OneSpan (VASCO) FM-enabled the token policy, and the token group with the OneSpan (VASCO) FM vendor present.
7. Click **Preview Token** to load the token. The list of the token serial number will be displayed if the OneSpan (VASCO) FM token is imported successfully.
8. Click **Import Token** to save the tokens.

Assign Token to User

Perform the following tasks to assign OneSpan FM tokens to the users for authentication with tighter security via SafeNet ProtectServer HSM.

Configure authentication realm(s)

Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** on the left pane. Use the pre-defined authentication realm "40192" for the Vasco token and assign it to the target user store.

> **Note**: Refer to *AccessMatrix Common Administration Guide - Configure Authentication Realm* section for more information.

Create user login module

Next, create the user's login module that uses the OneSpan token vendor "VASCO_FM_ERACOM".

1. Navigate to **Administration Dashboard > Configuration > Authentication > Login Module** on the left pane.
2. Click **Create** to go to the **Choose Login Module Implementation** page.
3. Select "Vasco Token Login" and click **Next** to the **Create Login Module** page.
4. Enter the **Login Module Id** and the **Login Module Name**.

5. Select the token vendor, "VASCO_FM_ERACOM", on the dropdown list.
6. Click **Save**. A new login module is created.

> **Note**: Refer to *AccessMatrix Common Administration Guide - Configure Login Module* section for more information.

Resynchronize the Tokens

Tokens will be out-of-sync with the AccessMatrix security server due to the time drift from the GMT time. It is, therefore, necessary to resynchronize the tokens before assigning them to the users. Resynchronize the token(s) at AccessMatrix server using the Admin Console. Refer to *UAS Administration Guide - Token Administration - Resync Token* section for more information.

Assign Token

1. Navigate to **Operation Dashboard > User & Segment > User**. Search and select the user to assign with a token.

   > **Note**: A new user can also be created if necessary. Refer to *AccessMatrix Common Administration Guide - User Management - Manage Users* for detailed information about creating new users.
2. Click the **Token Assignment** button and select a **Token Reassignment** option:
   - To allow a system to assign an available OneSpan token from the selected Token Group, select the **System Assigned** option, click **Search** and select the group from the **List Groups** drop-down list.
   - To manually assign a token to a user, select **Enter Serial Number/ID** option and enter the OneSpan token serial number. Select "VASCO" from the **Token Vendor** drop-down list.
3. Click the **OK** button to start the token assignment process. An information message for a successful token assignment will be displayed. Click **Yes** to continue the token assignment process; otherwise, click the **No** button.
4. The assigned token will be displayed under **User & Segment > User > Assigned Token** tab.

Assign Authentication Realm to User

For this example, assign the authentication realm "40192 (Vasco token)" to the user.

1. Navigate to **Operation Dashboard > User & Segment > User**.

2. Click the **Search** button to select an existing user, or create a new user.
3. On the **Search User Result** list, select an existing user.
4. Select **Login Accounts** tab, and click **Add**.
5. Select the authentication realm "Vasco Token (40192)" to assign to the current user.

> **Note**: The administrator can carry out important tasks associated with tokens such as token assignment/unassignment, token resync, token status change, token sharing, and others. Refer to *UAS Administration Guide - Token Administration* for detailed steps and information about these tasks.

Test OneSpan Functionality Module token with SafeNet HSM using VACMAN Controller

You have now finished integrating the OneSpan Functionality Module (FM) firmware with the SafeNet ProtectServer HSM. The following sections describe how to test your system.

> **Note:** To test the OneSpan FM login, you can use the usoagent or the AccessMatrix REST API.

For testing, first ensure that VACMAN Controller 3.22.0 has been installed in client machine. Next, perform the steps detailed in the following sections:

Prepare libraries and keys for testing

Navigate to the VACMAN Controller 3.22.0 installation folder at *C:\Program Files\VASCO\Authentication Server Framework 64 bit Thales ProtectServer HSM 3.22.0\Sample\java\Post-PTK4*. Follow the instructions in the **readme.txt** file:

1. Copy the following files:
    - Copy the **aal2sdk.dll** dynamic library from *C:\Program Files\VASCO\Authentication Server Framework 64 bit Thales ProtectServer HSM 3.22.0\Bin* to the current directory.
    - Copy the **aal2wrap.jar** java wrapper from *C:\Program Files\VASCO\Authentication Server Framework 64 bit Thales ProtectServer HSM 3.22.0\Wrappers\Java* to the *jar* folder within the current directory.
    - Copy the **jcprov.jar java** library (from the ProtectServer SDK) to the *jar* folder within the current directory.

- Copy the entire *C:\Program Files\VASCO\Authentication Server Framework 64 bit Thales ProtectServer HSM 3.22.0\Dpx* directory to the current directory.

2. Navigate to *C:\Program Files\VASCO\Authentication Server Framework 64 bit Thales ProtectServer HSM 3.22.0\Dpx* and open the **KeysDblEncryptDemo.txt** file. This text file will be referenced in the following steps.
3. Generate a storage key in the SafeNet ProtectServer HSM:
    a. Run the HSM KMU utility and select **Options > Create > Secret Key**.
    b. Set the **Label** of the Storage Key to "vascoStorageKey".
    c. Ensure that the **Wrap**, **UnWrap**, and **Encrypt** properties are selected.

    > **Note:** If you encounter the error "Error 913 - Invalid HSM Key Property" during the test, modify or recreate the key *without* the **Encrypt** property and try again.

    d. Click **OK** to finish the key generation.
4. Create the KEK (Key Encryption Key) from the components provided in the **KeysDblEncryptDemo.txt** file:
    a. Run the HSM KMU utility and select **Options > Create > Enter Key from Components**.
    b. Set the **Label** of the key to "KEK" and click **OK**.
    c. Specify "3" for **Number of components to enter**.
    d. Enter each of the 3 components provided in the **KeysDblEncryptDemo.txt** file when prompted.
    e. Complete the remaining steps to finish the key import.
5. Import the transport key to the SafeNet ProtectServer HSM:
    a. Run the HSM KMU utility and select **Options > Import Key(s)**.
    b. For **Unwrap Key**, select "KEK"
    c. Tick the **Import encrypted parts** checkbox and select the **Single Part** radio button.
    d. Click **OK**.
    e. Set the **Label** of the key to "vascoTransportKey" and click **OK**.
    f. Provide transport key's wrapped value when prompted to **Enter Hex Component**. This value is specified in the **KeysDblEncryptDemo.txt** file.
    g. Complete the remaining steps to finish the key import.

Prepare OneSpan hardware emulator

When running the test programs, you are required to generate OTP, challenge response or digital signature. To do so, you can use the hardware emulator provided by OneSpan:

1. Using a browser, access the hardware emulator at https://gs.onespan.cloud/te-demotokens/dp275.

   > **Note:** The DP275 token is recommended as it has all 3 of the functions (OTP/CR/SIGN) noted above.

Build and run the test

Before building and running the test, ensure that you are using slot 0 on the HSM for the test. This is because the test program has hard-coded the token slot to 0; using a different slot will lead to the error "Error 908 - Specified HSM Key not found". Alternatively, if a different slot must be used, you can update the **aal2tst.java** file of the test program to use the correct slot:

1. Navigate to *C:\Program Files\VASCO\Authentication Server Framework 64 bit Thales ProtectServer HSM 3.22.0\Sample\java\Post-PTK4\src* and open **aal2tst.java**.
2. Search for HSMSlotID.
3. Update the value of HSMSlotID from "0" to the correct slot.
   For example, if you are using slot 1, update the value as follows:

   Java

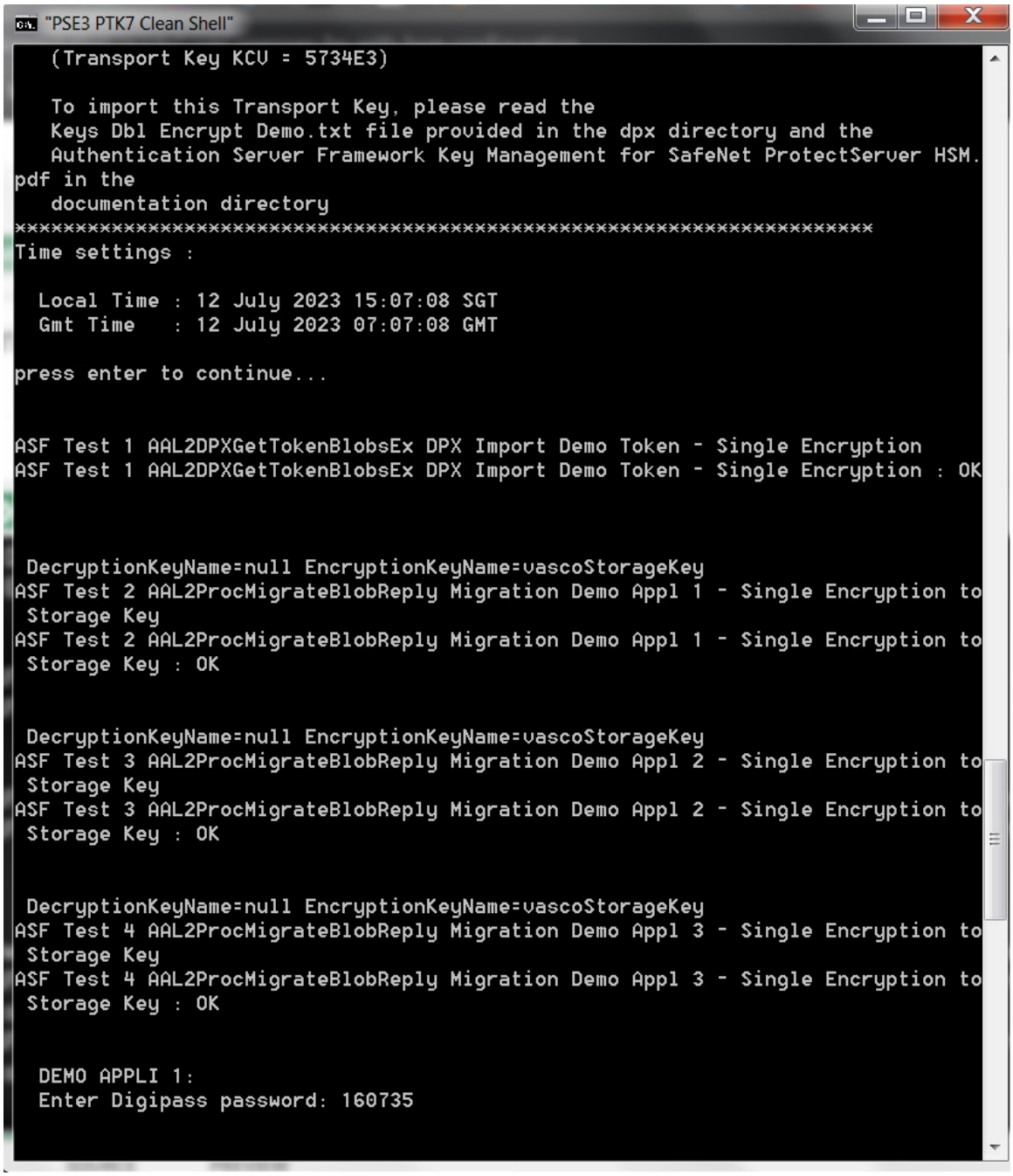
   ```
                   1,          // HSMSlotID - HSM Slot id
   used to store DB and Transport Key
   ```

4. Save the changes.

Once ensuring that the correct slot will be used for the test, you can now build and run the test. To do so:

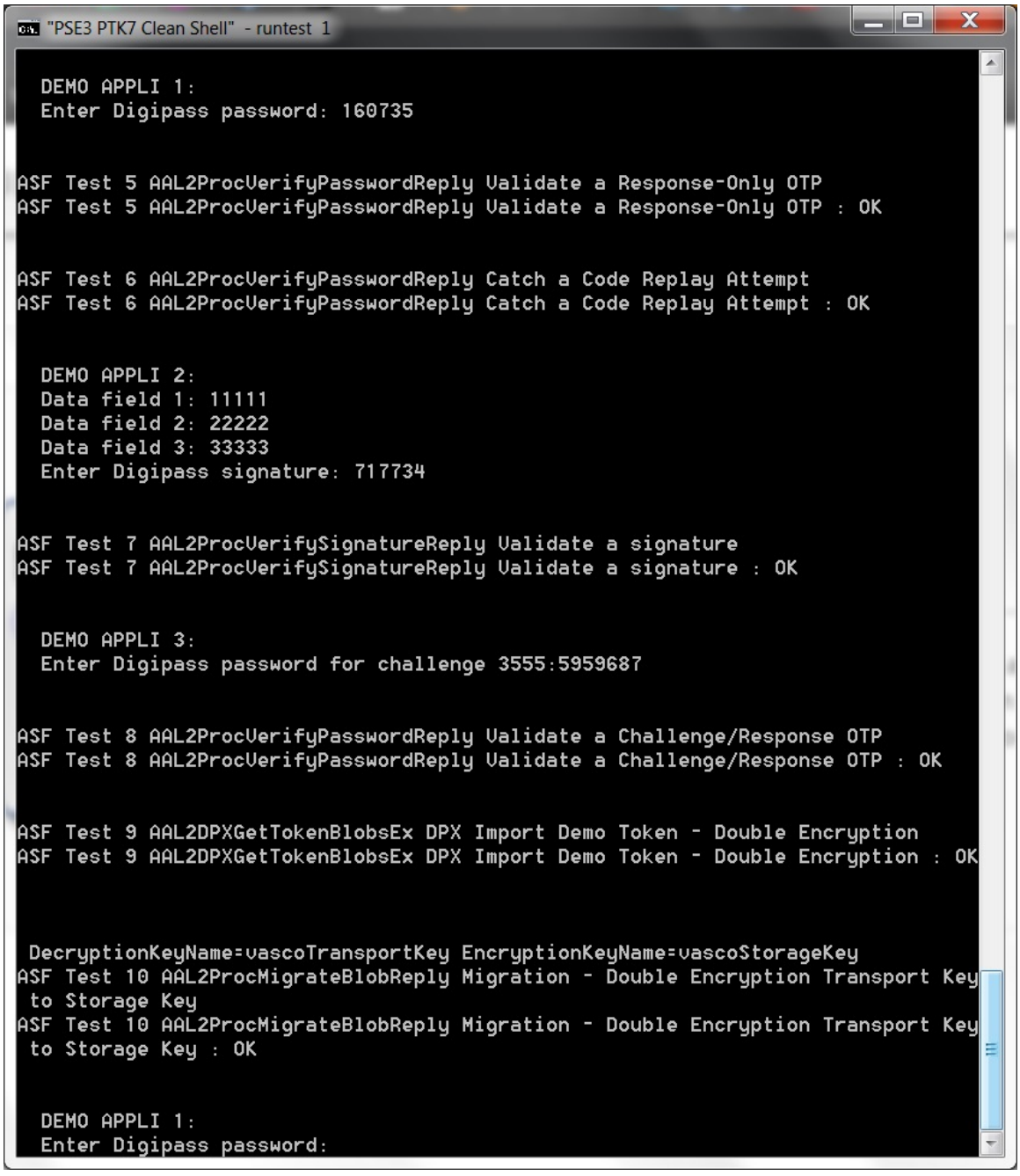
1. Open a command prompt window.
2. Browse to the path *.../sample/java* that contains the batch files created in the previous section (e.g. *C:\Program Files\VASCO\Authentication Server Framework 64 bit Thales ProtectServer HSM 3.22.0\Sample\java\Post-PTK4*).

3. Run **buildtest.bat**. If the java classpath directories have been set correctly, the command will not return any error.
4. Next, run **runtest.bat**. If there is no error in the java classpath directory, the following printout will be shown:



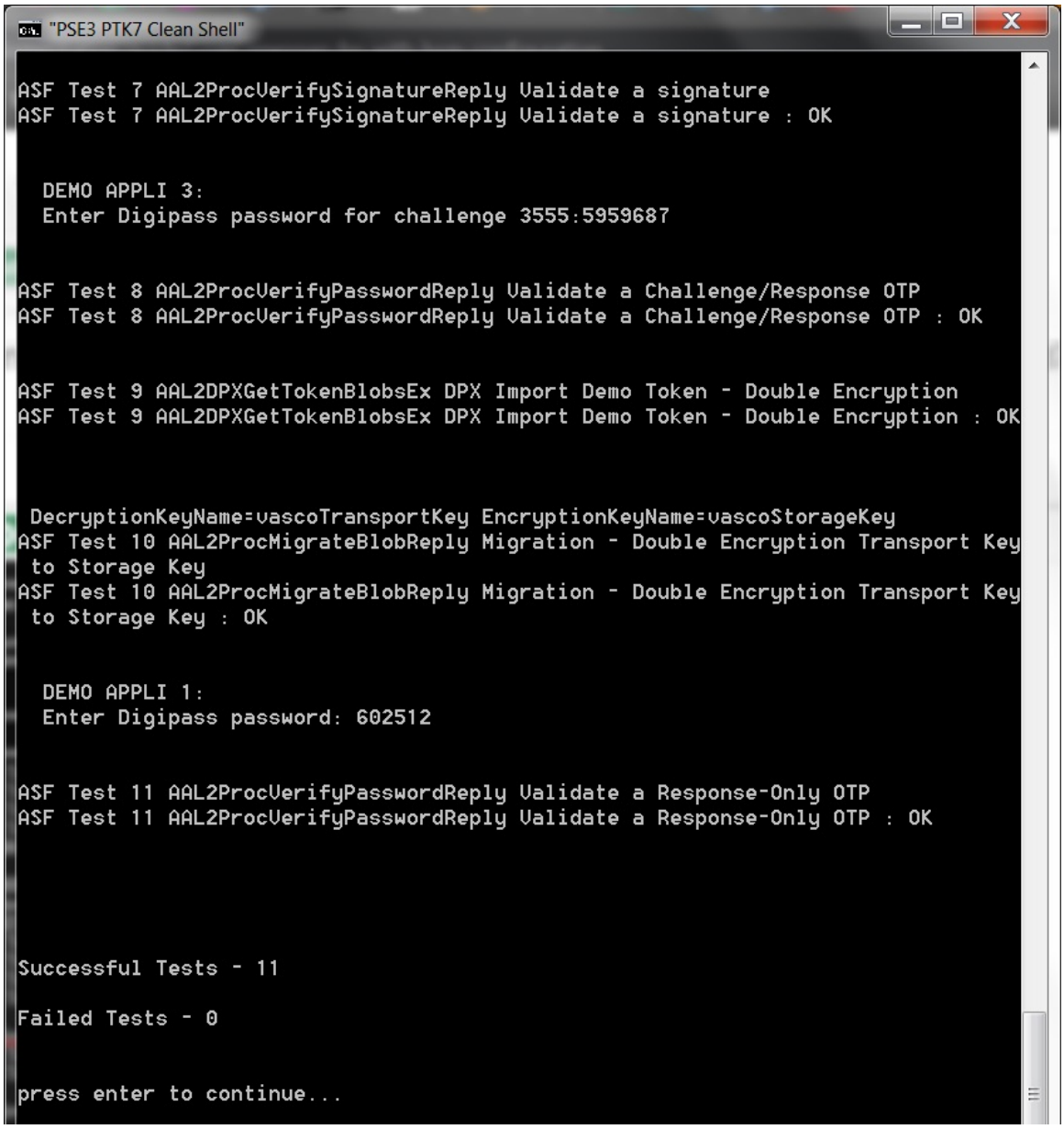
5. For the Test 1 - 4 printouts, ensure that `Single Encryption ...: OK`.
6. If testing on single encryption, the administrator is required to enter the Digipass password generated by any sample token. Otherwise, press Enter to skip this single encryption input.

7. Continue with the single encryption verification by entering the Digipass password (OTP), Digital Signature and Challenge password. The Test 5 - 9 printouts should show `Validate ...: OK`, etc..



8. Next, the test program will run Test 10 for double encrypted DPX. Ensure that the printout shows `:Double Encryption Transport Key to Storage Key: OK`.

9. Continue with the single encryption verification by entering the Digipass password (OTP). The Test 11 printout should show `Validate a Response-Only OTP: OK`.



10. Press Enter to exit the test program.

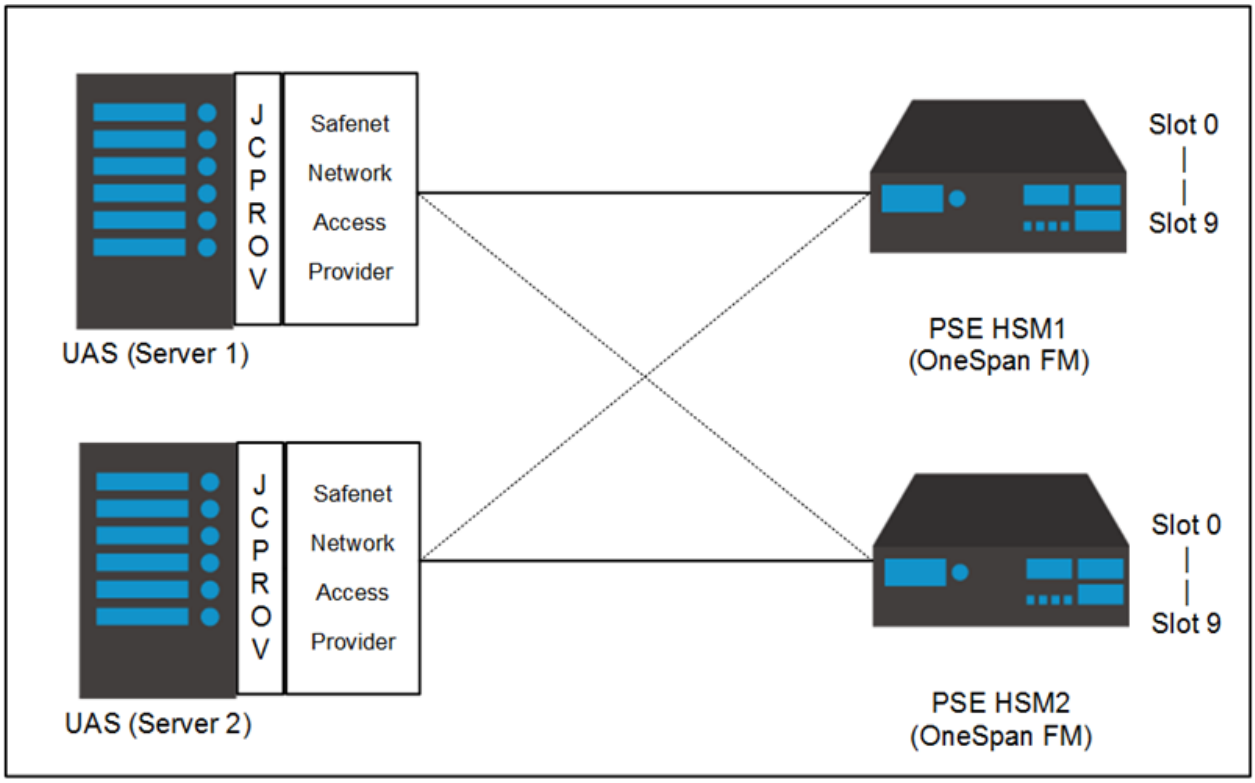
3.3. UAS OneSpan FM with SafeNet Jcprov Failover

AccessMatrix UAS uses Safenet JCProv API to integrate with OneSpan (VASCO) Token on Safenet ProtectServer HSM. Safenet Network Access Provider is used to access a remote HSM and it allows a configuration of multiple ProtectServer HSMs. The HSM IP Address and port configuration are stored as part of the Network Access Provider configuration.

The Jcprov API sees all of the combined slots of the HSMs (e.g. both HSM1 and HSM2 have 10 slots each, Jcprov API will see 20 virtual slots where virtual slot 0-9 correspond to HSM1 slot 0-9 and virtual slot 10-19 belong to HSM2 slot 0-9). The combining of slots happens on Jcprov API initialization. This means, based on the previous example, if during initialization only secondary HSM is up, Jcprov API will only return 10 slots (virtual slot 0-9).

For every AccessMatrix UAS request to HSM, AccessMatrix UAS will open an HSM session, do the HSM operation, and close the session. When opening a session, AccessMatrix UAS needs to specify the virtual slot to open. Thus, AccessMatrix UAS can choose which HSM to connect to by indicating the slot Id.

SafeNet does not support failover when FM is used and recommends i-Sprint to make use of the virtual slots as a way to achieve failover.



Failover Concept & Consideration

Approach

AccessMatrix UAS can choose which HSM to connect to only by specifying the slot Id. Based on the previous example (20 virtual slots, slot 0-19, correspond to HSM1 slot 0-9 and HSM2 slot 0-9), AccessMatrix UAS can connect to HSM1 slot 0 by specifying slot Id 0 and connect to HSM2 slot 0 by specifying slot Id 10.

The configuration of primary and secondary slots must be using slot/token label as slot Id may shift around. AccessMatrix UAS shall use slot/token label to find the primary and secondary slot Ids.

The failover can be achieved by first opening a session to a slot belonging to primary HSM. Then, do the required HSM-related operation. If somehow the operation fails with an error that is “unexpected by AM” (discussed at later point), then AccessMatrix UAS shall:

- Close the existing session to primary HSM
- Write to a specific log file with error code
- Open a new session to a slot belonging to secondary HSM and retry the required operation

When the problem on primary HSM has been rectified, and the HSM is brought online, Jcprov API does not automatically recreate the connection. For Jcprov to get the primary HSM connection again, Jcprov API needs to be reinitialized/restarted. The best and safest way to achieve this is to restart AccessMatrix UAS.

AccessMatrix UAS provides a feature to reinitialize Jcprov API without server restart; however, this process is disruptive and currently only supported for E2EEA module. Thus, this feature is not normally enabled. Jcprov reinitialization process requires all Jcprov-related activities to be halted. This process may take a few seconds, depending on the load at that time. This reinitialization feature should only be used when there is only one module in AccessMatrix UAS that makes use of the HSMs. For example, if AccessMatrix UAS has two modules, E2EEA and Key Management, that use any of the primary or secondary HSMs, then this reinitialization feature should not be used at all. Similarly, if AccessMatrix UAS has two E2EEA login modules, then the reinitialization feature should not be used either. And note that this feature is done on a best-effort basis to workaround on the Jcprov API limitation; thus, if Jcprov API still fails to re-establish the connection, then there is nothing that AccessMatrix UAS can do.

Conditions for Failover

The condition that triggers HSM failover is an error that is unexpected by AccessMatrix UAS, such that AccessMatrix UAS cannot fulfil any HSM-related request. This error is referring to the error returned by the HSM, not OneSpan (VASCO) or E2EE FM. Thus, HSM shall return OK (no error) when OneSpan (VASCO) or E2EE FM returns wrong OTP, out-of-sync, etc.

HSM errors that are expected by AccessMatrix UAS:

- CRYPTOKI_ALREADY_INITIALISED (expected during initialization)
- USER_ALREADY_LOGGED_IN (expected when AccessMatrix UAS is opening a session)
- USER_NOT_LOGGED_IN (expected when AccessMatrix UAS is doing HSM operation)

The failover conditions above are parameterized in the AccessMatrix UAS configuration file so that it is easily tuned in testing and production operation.

Failover Logic

> **Note**: This example is used to explain the failover logic.

HSM operation:

1. Open session to HSM (the active HSM)
2. Close session

If any unexpected error occurs on step 1 or 2 of HSM operation:

1. Print error to log/raise an alert
2. Make the next HSM in the list as the active HSM. If no more available HSM, then throw an exception to the caller
3. Redo from step 1 (open session)

Constraints & Assumptions

- Best if slot/token labels must be configured to be unique across HSMs. If not, then HSM serial number shall be used as the prefix of the label.

- (OneSpan (VASCO) FM only) OneSpan's (VASCO) kernel parameter requires AccessMatrix UAS to pass-in a slot index for the OneSpan (VASCO) FM to use the HSM key. This slot index (local/physical slot #, not virtual combined slot #) must be the same across all HSMs as OneSpan (VASCO) FM can see only the slots of the host HSM. That means if a slot for OneSpan (VASCO) FM operation is in slot 0 for HSM1, then on HSM2, it must be on slot 0 as well.

Configurations

SafeNet HSM Configuration

The timeout configuration determines how fast Jcprov returns an error to AccessMatrix UAS when there is a problem when invoking an HSM function. It is better to set it to a lower value, especially for the read timeout. However, the value may be affected by the environment. If possible, set the read and write timeout to 10-15 seconds.

- ET_HSM_NETCLIENT_SERVERLIST
    - Specifies multiple HSM IPs and ports (separated by space)
    - Jcprov orders the virtual slots based on the order specified in this configuration
- ET_HSM_NETCLIENT_READ_TIMEOUT_SECS (default value is 300)
- ET_HSM_NETCLIENT_WRITE_TIMEOUT_SECS (default value is 60)
- ET_HSM_NETCLIENT_CONNECT_TIMEOUT_SECS (default value is 60)

AccessMatrix UAS Configuration

More than one token label is required to configure the failover. Configuration of token labels depends on the modules.

Currently, the only way to configure is to add the am.crypto.vasco_fm.<token_policy_id>.hsmTokenLabels configuration in the **amsystem.properties** file.
For example, if the token policy Id is "VascoKernelParameterEracomHSM", the property configuration should be:

Properties (.properties files)

```
am.crypto.vasco_fm.VascoKernelParameterEracomHSM.hsmTokenLabels=VascoSlot
1, VascoSlot2
```

It is best to have unique token labels across multiple HSMs. However, if the token labels to be used are somehow not unique, it is still possible to configure failover by adding HSM serial number as a prefix to the token label: <hsm_serial_no>:<token_label>.
For example: 400081:VascoSlot, 400190:VascoSlot

Below are the HSM errors that are expected by AccessMatrix UAS (that will NOT trigger failover):

- CRYPTOKI_ALREADY_INITIALISED (expected during initialization)
- USER_ALREADY_LOGGED_IN (expected when AccessMatrix UAS is opening a session)
- USER_NOT_LOGGED_IN (expected when AccessMatrix UAS is doing HSM operation)

To override the list of expected error:

- Set am.crypto.jcprov.failover.ignored-error-codes in **amsystem.properties**.
- The error codes are Safenet HSM error codes. For multiple error codes, separate them by comma.

AccessMatrix UAS has a special log category for failover-related:

- com.isprint.am.util.JcprovUtil.Failover

3.4. UAS OneSpan FM with SafeNet ProtectServer HSM Troubleshooting and References

A. Common Errors for OneSpan (VASCO) - SafeNet HSM Integration

30112 PKCS11 Standard Error

This error may occur during token import or verification operations if the token policy was not configured properly or an unsupported OneSpan (VASCO) FM version was downloaded into the HSM. Perform the following steps if an error is encountered:

1. Check the token policy configuration and make sure the HSM slot Id is correct & available with corresponding login pin specified.
2. Try re-import token, if re-import encounter the same error, then check step 1 again and restart AccessMatrix UAS server.
3. If re-import was successful, try verification.
4. If the same error occurs then it could be that a wrong version FM was downloaded into the HSM.
5. Check the firmware version of HSM with this command: `ctstat`
6. Try to re-download the OneSpan (VASCO) FM because an unsupported version of OneSpan (VASCO) FM may have been downloaded into the HSM previously.
7. In OneSpan (VASCO) FM - SafeNet HSM VacmanController Package, you can find two **aal2sdk.signed** files inside:
    a. *VACMAN Controller Safenet HSM 3.18.0\Hsm\k5-arm*\
    b. *VACMAN Controller Safenet HSM 3.18.0\Hsm\k6-ppc*\
8. **aal2sdk.signed** in the *k5-arm* folder require that the SafeNet firmware version in the HSM is at least version 2.01.00.
9. **aal2sdk.signed** in the *k6-ppc* folder require that the SafeNet firmware version in the HSM is at least version 5.00.02.
10. Therefore, solve this error by downloading the **aal2sdk.signed** with matching firmware version into the HSM.

B. Utility Reference - SafeNet HSM ProtectToolkit C Runtime

Utility	Description
CTCERT	Certificate Management Utility provides basic support for the creation of X.509v3 certificates using ProtectToolkit C product. Use this utility to: • Generate both self-signed certificates and certificates signed with a specified CA key. Command: `c` For example: `ctcert c` • Generate PKCS #10 certificate requests. Command: `r` For example: `ctcert r` • List certificates, certificate requests, and key objects that exist in a specified slot. Command: `l` For example: `ctcert l` • Import certificates. Command: `i` For example: `ctcert i` • Export certificates. Command: `x` For example: `ctcert x` • Mark certificates as trusted. Command: `t` For example: `ctcert t`
CTCONF	Configuration Utility is used to configure the operating parameters for ProtectToolkit C. It will report configurable settings for the first device found.
MKFM	It is used to make a timestamp, hash and sign an FM binary image.

CTCERT Utility	Synopsis
ctcert c	[-s<slot>] [-i<slot>] [-c<label> |-k] [-t<type>] [-z< bits>][-x< name>] [-b <YYYYMMDDhhmmss[Z]>] [-e <YYYYMMDDhhmmss[Z]>] [-d <duration<h|d|m|y>>] [-C<curve_name>] [-S<mechanism>] -l<label>
ctcert i	[-s<slot>] [-f<file>] -l<label>
ctcert l	[-s<slot>]
ctcert r	[-s<slot>] [-k] [-t<type>] [-z< bits>] [-f< name>] [-S<mechanism>] -l<label>
ctcert t	[-s<slot>] -l<label>
ctcert x	[-s<slot>] [-f< name>] -l<label>

CTCONF Utility	Synopsis
ctconf	[-a<device>] [-b<name >] [-c<slots>] [-d<slot>] [-e] [-f<flags>] [-g<file>] [-h] [-i<file>] [-j<file>] [-k<file>] [-l] [-m<mode>] [-n< slot >] [-p] [-q] [-r<slot>] [-s] [-t] [-v] [-x] [--rtc-adj-access-control-rule = [secs]:[count]:[days]] [--rtc-adj-access-control = 0|1]

MKFM Utility	Synopsis
mkfm	[-f<filename>] [-k<key>] [-o<filename>] [-3] **Note:** `-o` denotes the output filename, and `-3` is required only when you want to sign a new FM with an existing certificate that was created for use with Protect Processor Orange 3 (PTK 3).

Note: Refer to VACMAN Controller SafeNet HSM documentation, *VACMAN Controller HSM Module Management For Safenet ProtectServer HSM*, for more information.

4. UAS OneSpan Token Configuration (Non-HSM)

Setup Steps

This section defines the steps relating to the configuration of the UAS OneSpan Token solution.

- **Configure the OneSpan Authentication Server Framework library**
  Before OneSpan tokens can be used with AccessMatrix UAS, OneSpan Authentication Server Framework must be properly installed. Authentication Server Framework is a unique and flexible platform that supports multiple authentication devices and mechanisms. Refer to Configure OneSpan Token Libraries for more details.
- **Configure the global setting for token management**
  The administrator can use the Token Management in the Global Setting to modify the Token Import Utility (TIU) features and token functions for various OneSpan token models. To access this page, in the Admin Console, navigate to **Administration Dashboard > Configuration > Global Setting** and click the **Token Management** tab. Modify the Token Management attributes such as **Token Status Unassign**, **Token Status (if threshold exceeded)**, and other hardware token-related attributes based on requirements. Refer to *UAS Administration Guide - Token Management* for additional details.
- **Setup Token Import Utility (TIU) Configuration**
  Token Import Utility (TIU) configuration is where the administrator defines the list of token groups available for selection during the token import process, whether or not overwriting of an existing token is allowed, as well as the maximum tokens for token group reassignment. To configure, navigate to **Administration Dashboard > Configuration > Token > TIU Configuration** and click the **Edit** button. Modify the attributes based on the requirements. Refer to *UAS Administration Guide - TIU Configuration* for more details.
- **Configure the OneSpan token policy**
  OneSpan token policy manages the behavior of the OneSpan hardware token used for authentication. This policy is to be used during login module administration. If the OneSpan Authentication Server Framework libraries are already placed in AccessMatrix UAS, a predefined OneSpan token policy called "DefaultVascoKernelParameters" will be created upon successful restart of AccessMatrix UAS. However, a new token policy can also be created. Refer to Configure OneSpan Token Policy for policy details.
- **Configure the token batch**
  Token Batch refers to the physical grouping of token devices. Creation of token batch can be

done in the Admin Console or during token import using Token Import Utility (TIU). Each batch is tied to a particular token policy set for a specific vendor, e.g. the OneSpan Token Batch needs to be configured to use the token policy predefined for OneSpan (i.e. "DefaultVascoKernelParameters"). Refer to Configure Token Batch for configuration details.

- **Configure the token group**

  Token Group refers to the logical grouping of OneSpan token devices. Creation can be done in the Admin Console or during token import using TIU. Refer to Configure Token Group for details.

- **Import the OneSpan token**

  Before a token can be used, it must be imported to the AccessMatrix system first via Token Import Utility (TIU), an application used for bulk import of tokens in the database. Before importing, secure the OneSpan DPX file and the associated transport key of the tokens. The DPX file contains logical token information which includes token seed and token serial number information. Refer to Import OneSpan Token for the import steps.

- **Configure Login Module**

  Vasco Token Login is a OneSpan token-specific login module that provides OneSpan token-based login method for the user who may reside in either an internal or external user store. A predefined VASCO token login module called "DefaultVascoTokenLogin" will be created upon installation. However, a new login module can also be created. Refer to Configure Login Module for the configuration steps.

- **Configure Authentication Realm**

  Authentication Realm associates user credentials for authentication based on a lookup module and login module(s) where user entries can be created into a local repository of AccessMatrix or integrated from LDAP (Lightweight Directory Access Protocol). By default, a predefined OneSpan authentication realm called "40192" will be created upon successful installation of AccessMatrix. However, a new authentication realm can also be created. Refer to Configure Authentication Realm for configuration details.

- It is assumed that the lookup module is already configured and pointing to the user store. Refer to *AccessMatrix Common Administration Guide - Configure User Lookup Module* for configuration details. Another login module may also be created and assigned to the same authentication realm depending on the use case requirements.

- **Assign login accounts to the user**

  A user is an entity that accesses the system and may belong to a default store (local repository)

or an external user store, e.g. AD (Active Directory). Assign the configured authentication realm to the user by adding it to the **Login Accounts** tab. Refer to Configure User Accounts for details.

- **Assign OneSpan token to the user**

  The token can be assigned and unassigned in the Admin Console. It can be either through system assignment or by manually entering the token serial number. Refer to Assign Token to User for details.

Configure OneSpan Token Libraries

Ensure that OneSpan Authentication Suite Server SDK (A3S) has been installed. Next, retrieve the following files from the A3S installation directory and copy them to the AM5 folder. For example:

> **Important Note:** A3S upgrades may not be backward compatible. Before attempting an upgrade, ensure that you read *AccessMatrix Product Upgrade Guide - Breaking Changes in OneSpan Authentication Suite Server SDK Upgrade*.

Windows	• **aal2sdk.dll** - from *<InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Bin*, copy to *<AM5InstallationDirectory>\tomcat\bin*. • **aal2wrap.jar** - from *<InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Wrappers\Java*, copy to *<AM5InstallationDirectory>\tomcat\webapps\am5\WEB-INF\lib*.
Linux	• **libaal2sdk.so** - from *<InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Bin*, copy to the directory specified in the machine's java.library.path. • **aal2wrap.jar** - from *<InstallationDirectory>\OneSpan\Authentication Suite Server SDK 64 bit 4.0.1\Wrappers\Java*, copy to *<AM5InstallationDirectory>\tomcat\webapps\am5\WEB-INF\lib*.

> **Notes**:
>
> - If the JAVA version is 64-bit, ensure that all the libraries used are in a 64-bit version.
> - AccessMatrix UAS can be started once all the libraries are being placed correctly. Monitor **am.log** for any errors during the server startups. If any library is not compatible, the VASCO token vendor handler will not be registered/initialized successfully and hence, it cannot be used.

Configure OneSpan Token Policy

To configure the OneSpan token policy, navigate to **Administration Dashboard > Security Policy > Token** and click **Token Policy** on the left pane. Select the pre-created token policy "DefaultVascoKernelParameters". Alternatively, click **Create**, select the "VASCO Kernel Parameters"

implementation from the **Choose Token Policy Implementation** page and click **Next** button. Then, specify the details for the token policy attributes before saving the changes.

> **Note**: The OneSpan attributes of this policy will only be displayed if the OneSpan Authentication Server Framework libraries are already placed in AccessMatrix UAS.

Attributes	Description
Attributes Tab	
OneSpan (VASCO) Specific	
Storage Derive Key 1	Four random numbers (Integer) are required by OneSpan to generate secure storage keys. (Uneditable)
Storage Derive Key 2	
Storage Derive Key 3	
Storage Derive Key 4	
Check Challenge	Verify or not if the challenge has been corrupted before validation. • "0" - No password checking • "1" - Check parameter then verify with the DPData Challenge • "2" - Always use the DPData Challenge to validate responses • "3" - Avoid Challenge-Response Replay Attack by allowing only one Challenge-Response authentication per timestep • "4" - Avoid Challenge-Response Replay Attack by rejecting the second response if responses from 2 consecutive authentication requests are equal and in the same timestep Default: 0
Check Inactive Days	Acceptable number of days of user/token inactivity. If days of inactivity is higher than the specified number, then 205 return code will be generated and the Digipass will have to be reset. "0" means this feature is disabled. Range value: 0 to 1024 Default: 0
Derive Vector	The vector is used to make data encryption unique for a host. Range value: 0x00000000 to 0x7fffffff Default: 0
Diag Level	The level of diagnostic messages generated by functions, which goes to standard output.

Attributes	Description
	Range value: 0 to 3 Default: 0
Event Window	Event Window size, expressed in a number of iterations. This represents the acceptable event counter difference between DIGIPASS and host. This parameter applies only to event-based operating modes. Value should not be less than 10. Range value: 10 to 1000 Default: 100
GMTAdjust	GMT Time adjustment to perform in case the C language gmtime function does not give an accurate value. Range value: -86400 to 86400 Default: 0
HSM Slot Id	Stores Storage Key and Transport Key. It is applicable only for HSM integration. Range value: 0 to 60 Default: 0
IThreshold	Number of successive Identification errors allowed. When the specified number is reached, a 202 return code is sent to the caller. The token will then need to be resynchronized. 0 means this feature is disabled. Range value: 0 to 255 Default: 3
ITime Window	The number of timesteps that determines the acceptable time difference between a Digipass and the host system for the Identification function. This difference is adjusted to the last known shift for each token. The maximum drift correction acceptance is 1 second per 6 hour period. The time step is determined by the Digipass internal programming options. Value should not be less than 2. Range value: 2 to 1000 Default: 10
ITime Window 1	Additional ITime Window values if the client wanted to use a different time window during run time. A user can specify the ITime Window to follow during API calls.
ITime Window 2	
ITime Window 3	
Online SG	The level of Online Signature. • "0" - Disabled feature • "1" - Several Signatures are allowed in the same TimeStep (except identical successive ones) • "2" - Only one signature per timestep allowed Default: 2

Attributes	Description
SThreshold	The number of successive Signature errors allowed. When the specified number is reached, a 203 return code is sent to the caller. The token will then need to be resynchronized. 0 means this feature is disabled. Range value: 0 to 255 Default: 3
STimeWindow	The number of timesteps that determines the acceptable time difference between a Digipass token and the host system for the Signature function. This difference is adjusted to the last known shift for each token. The time step is determined by the Digipass internal programming options. Value should not be less than 2. Range value: 2 to 500 Default: 10
STimeWindow 1	Additional STime Window values if a client wanted to use different time windows during run time. The user can specify the STime Window to follow during the API calls.
STimeWindow 2	
STimeWindow 3	
Storage Key Id	The key Id used to read (Decrypt) Digipass Blob from the database. Range value: 0 to 0xffffffff Default: 0
Synchronize Window	For InitialTimeWindow Configuration. This parameter sets the initial derivation between a DIGIPASS time and the GMT Time. This value is expressed in hours. Value should not be less than 1. Range value: 1 to 60 InitTimeWindow=(max(STimeWindow or ITimeWindow), SyncWindow) Default: 2
Synchronize Window 1	Additional Synchronize Window values if the client wanted to use a different time window during run time. The user can specify the Synchronize Window to follow during API calls.
Synchronize Window 2	
Synchronize Window 3	
TransportKeyId	Key Id used to write (Encrypt) Digipass Blob to the database. Range value: 0x00000000 to 0x7fffffff Default: 2
Preferred application ID for	Specify the preferred OneSpan application Id of the token blob to be used for OTP(RO) verification.

Attributes	Description
OTP (RO) if more than one blob	
Return activation password or authorization code to client	Define whether to return the generated activation password or authorization code to the client. Default: false
Secure Channel Application ID Keyword(s)	Specify the Application ID Keyword(s) for Secure Channel application used for activation that matches the Application ID(s) defined on OneSpans's parameter sheet.
Secure Channel Message Validity Time	This value indicates the Secure Channel(SC) Message validity time in seconds. During Secure Channel verification, this validity time is used to check whether the generated Secure Channel message has expired. The default value means this feature is disabled. Default: 0
Secure Channel Message Show MAC	If set to "true", the client device or application must present the calculated MAC to the user. Default: true
Secure Channel Message Show Warning	If set to "true", the client device or application must display a predefined warning message to the user. Default: true
Secure Channel Message Ask Approval	If set to "true", the client device or application must request the user to confirm the transaction data before the MAC is generated. Default: true
Secure Channel Message Ask PIN	If set to "true", the client device or application must request PIN verification before generating the MAC. Default: false
Secure Channel Message Show Data	If set to "true", the client device or application must display the content of the transaction message. Default: true
Number of Active Instances for Multi-Device Token	Specify an integer to limit the number of active instances for multi-device token. Enter one or more to allow one or multiple instances to be activated at one time. "0" or 'empty' means there is no limit to the number of active instances for multi-device token.
Post-Activation Cooldown Period	Specifies the duration (in seconds) where a newly activated instance will remain unusable.
Digipass for APPS/Mobile Specific Tab	

Attributes	Description
Digipass for APPS/Mobile Specific	
Server Activation Challenge Response	Enable Server Activation Challenge-Response if set to "true". This feature will return a challenge meant for the mobile application. The mobile application will then generate a response for that particular activation. Default: false
Score Based Response	Enable Score Based Response (contextual) verification if set to "true". This feature will perform verification on OTP response which contains User, Context and Platform scoring generated by the user's OneSpan token. Default: false
Generate & Send Mobile Activation QRCode After Assignment	This feature is strictly for Mobile Tokens only. If enabled, the server will generate Mobile Activation QRCode upon successful token assignment. This Activation QRCode will be sent to the user through an event notification policy. Please select the corresponding event notification policy in the **Event Notification Policy for Activation QRCode** attribute. Default: false
Include Activation Password in QRCode	Set to "true" to embed Activation Password into the Online Activation QRCode. Otherwise, configure the event notification policy to deliver the Activation Password. Default: true
Include Authorization Code in QRCode	Set to "true" to embed the Authorization Code into the Online Activation QRCode. Otherwise, configure event notification policy to deliver the Activation Password. Default: true
Event Notification Policy for Mobile Activation QRCode	Specify the event notification policy to be used for sending the Mobile Activation QRCode.
Allow Activation Checking	If set to "true", special validation will be performed during Digipass Mobile token activation to check if the token's "Allow Activation/Reactivation" flag is set to "YES". If the token's "Allow Activation/Reactivation" flag is set to "NO", the token activation will be denied. Set to "false" to disable this validation. Default: false
Allow API For Decrypting Secure Channel Message	If set to "true", the REST API for decrypting the secure channel message will be allowed.
Digipass Online Activation Secured Data Exchange Protocol	
Secured Data Exchange Protocol	Specify the type of protocol to be used for Secured Data Exchange during Digipass for APPS/Mobile online activation. • "SRP" - Secure Remote Password protocol

Attributes	Description
	• "ECDH" - Elliptic curve Diffie-Hellman protocol Default: ECDH
SRP Activation Specific	
User Password Length	SRP's user password length requirement. Default: 8
User Password Character Sets	SRP's user password character sets requirements. Default: Numeric
User Password Letter Case	SRP's user password letter case requirement. Default: Mixed Case Alphanumeric
Checksum on User Password	If set to "true", OneSpan SDK will calculate the checksum of the user's password. Default: true
User Password Validity Time (Seconds)	SRP's user password validity time. Default: 86400
ECDH Activation Specific	
Activation Password Length	ECDH's activation password length requirement.
Activation Password Character Set	ECDH's activation password character requirement.
Activation Password Validity Time (Seconds)	ECDH's activation password validity time.
Authorization Code	
Authorization Code Length	Authorization code's length requirement. Default: 6
Authorization Code Character Set	Authorization code's character set requirement. Default: Numeric
Authorization Code Validity Time (Seconds)	Authorization code's validity time. Default: 86400
Authorization Code Retry Threshold	Maximum failed attempts for Authorization Code verification during Activation. The Authorization Code will be invalidated upon the threshold

Attributes	Description
	reached. Set to "0" to disable this feature. Default: 3
CrontoSign Configuration	
Use Dynamic CrontoSign	Setting to "true" will enforce the use of dynamic size CrontoSign image generation to support up to 1070 characters for Secure Channel transaction signing data. If disabled, standard size CrontoSign(support up to 200 characters) will be used instead. Default: false
Encrypted DPX Transport Key Tab	
HSM Slot Id	HSM's slot Id.
HSM User PIN	HSM's user pin.
Transport KEK label	Label of transport KEK stored in HSM.

Configure Token Batch

To configure a token batch, navigate to **Operation Dashboard > Token > Token Batch** and click the **Create** button and provide the details for the token batch attributes before saving the changes.

Attribute Name	Description
*Token Batch Id	Identification assigned to the token batch.
*Token Batch Name	The name of the token batch.
Description	Brief description for the token batch.
Attributes Tab	
*Token Policy	Specify the OneSpan token policy, e.g. "DefaultVascoKernelParameters".
Static Vector	(Not applicable)
Manufacture Time	The date/time when the token is manufactured.
Expiry Time	The token expiry date/time.
Token Attributes Template Id	The identification of the template is used to set token attributes for additional custom information.

Configure Token Group

To configure a token group, navigate to **Operation Dashboard > Token > Token Group** and click the **Create** button and provide the details for the token group attributes before saving the changes.

Attribute Name	Description
*Token Group Id	Identification assigned to the token group.
*Token Group Name	The name of the token group.
Description	Brief description of the token group.
*Token Vendor	The token vendor to manage in this group. Ensure that the correct token vendor (i.e. VASCO) is configured to group similar tokens.
Segment	Name of the segment the token group belongs to.
Attributes Tab	
System Token Assignment	
The system by default will assign the oldest token first based on the time of a token is being placed in a token group. It is particularly useful to hardware tokens that have an expiry date due to the lifespan of the battery. For soft tokens, this is no longer required. It has a higher rate of token assignment as users change their devices such as mobile phones, etc. more often than hardware tokens. A soft token vendor such as V-Key sometimes requires the new tokens to be re-provisioned when they update their firmware. To allow higher system-token assignment capacity and to increase performance, it is highly recommended to set **Randomize Reservation Pool** to **True** and **Database Fetching Order of Reservation Pool** to **None**. These settings fetch the token reservation pool from the database quicker and reduce the chance of conflict and clashing during the token assignment for a more efficient system-token-assignment process. If the system expects a more extensive concurrent system-token-assignment request, increase the value of **Reservation Pool Size**.	
Randomize Reservation Pool	This is to randomize the available tokens when assigning to a user if there are large system token assignment concurrent requests. Select the setting "true" for soft tokens that may require token re-provisioning due to soft token SDK/firmware upgrade. It will fetch the token reservation pool from the database much quicker and reduce the chance of conflict and clashing during the token assignment. Default: false
Reservation Pool Size	Set the reservation pool size for random system token assignment. When handling high concurrent requests, set "Randomize Reservation Pool" to "true" and then set this pool size to at least 20. Default: 10

Attribute Name	Description
Database Fetching Order of Reservation Pool	Fetches from the database to get the reservation pool, this setting specifies the sorting of fetch. Select the setting "None" for soft tokens that may require token re-provisioning due to soft token SDL/firmware upgrade, such as V-Key soft Tokens. Setting the order to "None" will reduce the database processing tremendously. Default: With Ordering

Import OneSpan Token

To launch Token Import Utility, close the Admin Console login page and navigate to **Token > Token Import Utility**. Enter a token admin's login Id and password for **Login (User1)** (e.g. "tokenAdmin1") and another token admin for **Login (User2)** (e.g. "tokenAdmin2"). The TIU page will be displayed after a successful login.

Import steps:

1. On the TIU page, click the **Import Token** button.
2. Select an **Import Action** option. By default, the "Token Seeds Import" option is selected.
    a. Select **Token Seeds Import** to perform a normal token import.
3. Specify the OneSpan DPX file in one of the following ways:
    - **Upload local file** - Click the **Choose file** button and select the OneSpan DPX file to be uploaded, then press the **Upload** button.
    - **Or specify server file under system temp folder** - Perform the following steps:
        a. Place the DPX file into your AccessMatrix server's *temp* folder. This is typically located at *<ACMATRIX_ROOT>/tomcat/temp/*.
        b. Next, specify the name of the DPX file only (e.g. "example.dpx"). You do not need to specify the path to the temp folder.
4. Set up token batch.
    a. Select the OneSpan Token Batch from the drop-down list. Refer to the steps described in the Configure Token Batch section on how to create a token batch using the Admin Console.

b. OR create a new token batch via TIU, if necessary. Click the **Create** button to display the **Create Token Batch** dialog box. Specify the **Token Batch Id**, **Token Batch Name**, and **Token Policy** values and click the **Submit** button.

> **Note**: Select the token policy with "VASCO Kernel Parameters" implementation.

5. Set up token group.

   a. Select the "OneSpan" **Token Group** from the drop-down list. Refer to the steps described in the Configure Token Group section on how to create a token group using Admin Console.

   b. OR create a new token group via TIU, if necessary. Click the **Create** button to display the **Create Token Group** dialog box. Specify the **Token Group Id**, **Token Group Name**, **Token Vendor**, **Description**, and **Segment** values, if necessary. Click the **Submit** button.

   > **Note**: Select "VASCO" as Token Vendor value.

6. Select the default **Token Status** of tokens after import. Default value is "Activated".
7. Enter the **Transport Key**.

   > **Note**: KCV and key count are read-only and auto-generated by the system.

8. Specify the action on conflict. For cases where the token already exists in the system, the user can either overwrite existing tokens or terminate the token import activity.
9. [OPTIONAL] Enter the job description where the user can describe more about the current importing job.
10. [OPTIONAL] Click the **Preview Token** button to preview the tokens to be imported. Once clicked, a list of tokens with their details will be displayed.
11. Click the **Import Token** button to start the importing token process in the bulk job. Bulk job process is an asynchronous process that will be running in the background. A user need not wait for the job to finish and once importing token process is started, the system will prompt a dialog box allowing the user to view the Bulk job detail.

Once the token import is successful, the tokens can now be used for any authentication or authorization purposes.

Configure Login Module

Navigate to **Administration Dashboard > Configuration > Authentication > Login Module** and select the pre-created login module: "DefaultVascoTokenLogin" or create a new login module and select "Vasco Token Login" implementation. Click the **Next** button and provide the details for the login module attributes before saving the changes.

Attribute Name	Description
*Login Module Id	Unique Id of login module.
*Login Module Name	Name of login module.
Description	Description of login module.
Version	Login module entry version.
Implementation	Type of login module implementation, i.e. "Vasco Token Login".
Handler	Handler of the login module, i.e. "VascoTokenLogin".
Configuration Tab	
Login Configuration	
Login Policy	The login policy specifies if bad login count should be incremented and if login account should be locked upon exceeding maximum bad login count, as well as other policy controls. If no login policy is attached, then the corresponding policy controls like incrementing bad login count will not take effect.
Authentication Level	Unused
Token Vendor	
Token Vendor	Refers to the OneSpan (VASCO) token vendor. Default: VASCO
Default Event Notification Policy	
Event Notification Policy	It is an event-based trigger for sending email or SMS. For example, sending OTP to login users when Generate OTP event is triggered. **Note:** This is only the policy that will be invoked when resetting a password.
Retry Configuration	

Attribute Name	Description
Auto Retry	Setting to true will perform auto-retry to every token(s) assigned to the user. **Notes** (if set to "true")**:** • Only "Response-Only(RO)" mode will be functional for this login module. • Bad login count (IThreshold) checking has to be disabled in the corresponding OneSpan (VASCO) Token Policy. Default: false

Configure Authentication Realm

Navigate to **Administration Dashboard > Configuration > Authentication > Authentication Realm** and select the pre-created login module: "40192" or create a new authentication realm and provide the details for the authentication realm attributes before saving the changes.

Attribute Name	Description
*Authentication Realm Id	Unique Id of authentication realm.
*Authentication Realm Name	Name of authentication realm.
Description	Description of authentication realm.
Version	Authentication realm entry version.
Configuration Tab	
Automatically assigned to users	Allow this authentication realm to be automatically assigned to all users.
User Lookup Module	Lookup module is used to validate user identity (i.e. user search in the user store(s) or other validation methods).
First(1st) Login Module	The first login method to use for user authentication.
Second(2nd) Login Module	Optional subsequent user authentication method.
Third(3rd) Login Module	Optional subsequent user authentication method required.
Fourth(4th) Login Module	Optional subsequent user authentication method required.

Attribute Name	Description
Fifth(5th) Login Module	Optional subsequent user authentication method required.
Enable Login/Logout History Saving	Set this to false to increase performance if tracking of login/logout history is not required in your project or during a stress test. Default: true
User Response	On successful authentication, return name-value pair responses specified in this User Response.
Realm-Based Session Policy	The realm-based session policy of this authentication realm. If no realm-based session policy is assigned, then the session policy defined in **Global Setting** will be used.

An authentication realm can be configured to provide a chained authentication known as multiple-factor authentication (MFA) of up to five factors of authentication. It can either be configured as a 2-step or 1-step authentication depending on the login module configured. 2-step authentication is where a username and static password are sent for authentication first before prompting the user to enter the OneSpan OTP. Whereas, 1-step authentication is where password and OneSpan OTP are combined and submitted together during authentication.

The 2-step authentication can be set by creating an Authentication Realm where the Second(2nd) Login Module is set to use **Vasco Token Login** login module, e.g. "DefaultVascoTokenLogin". The 1st Login Module can be set to use "System Password Basic" or "Ldap Login".

On the other hand, the 1-step authentication can be set by creating an Authentication Realm where the First(1st) Login Module is set to use a **Composite Login** login module. This login module can support a combination of a dynamic password (a.k.a. OTP) and static password in the form of <OTP><Static Password> or <Static Password><OTP>.

Configure User Accounts

A user may belong to a default store or external user store. Refer to *AccessMatrix Common Administration Guide - Manage User Stores* to know about the different types of user stores and how to configure it.

To create a new user in the default store, navigate to **Operation Dashboard > User & Segment > User** and click **Create** on the right pane. Provide values to the following attributes that are necessary for the OneSpan token implementation before saving the changes.

Attribute Name	Description
*User Id	Unique user Id.
*User Name	Name of user.
*Interactive	Set to "Yes" to make it an interactive user.
User Store	Name of User Store the user belongs to, i.e. "Default Store".
Login Accounts Tab	
This tab contains the authentication realm and login module assigned to the user. Click **Add** to display the **Choose Authentication Realm** dialog box. Select the authentication realm created in the previous section and click **OK**. Both the authentication realm and the login module(s) associated with it will be assigned to the user.	

Refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

Assign Token to User

To assign tokens:

1. Navigate to **Operation Dashboard > User & Segment > User**. Search and select the user to assign with a token.

   > **Note:** A new user can also be created if necessary. Refer to *AccessMatrix Common Administration Guide - Manage Users* for detailed information about users.

2. Click **Token Assignment** button and select a **Token Reassignment** option:
   a. To allow a system to assign an available OneSpan token from the selected **Token Group**, select the **System Assigned** option and select the group from the **Token Group** drop-down list.
   b. To manually assign a token to a user, select the **Enter Serial Number/ID** option and enter the OneSpan token serial number. Select "VASCO" from the **Token Vendor** drop-down list.
3. Click **OK** button to start the token assignment process. An information message for a successful token assignment will be displayed. Click **Yes** to continue token assignment, otherwise, click the **No** button.
4. The assigned token will be displayed under **User & Segment > User > Assigned Token** tab.

Token Administration

The administrator can carry out important tasks associated with tokens such as token assignment/unassignment, token resync, token status change, token sharing, and others. Refer to *UAS Administration Guide - Token Administration* for detailed steps and information about these tasks.

5. UAS OneSpan Token Implementation Using API

The following pages provide detailed information on the various UAS OneSpan Token APIs:

- Login
- Token Verification
- Token Management
- Errors

5.1. Login

OneSpan tokens can also be generated and managed directly via API. The API supports token generation, verification, assignment/unassignment, changing of token status, resyncing of tokens, and others.

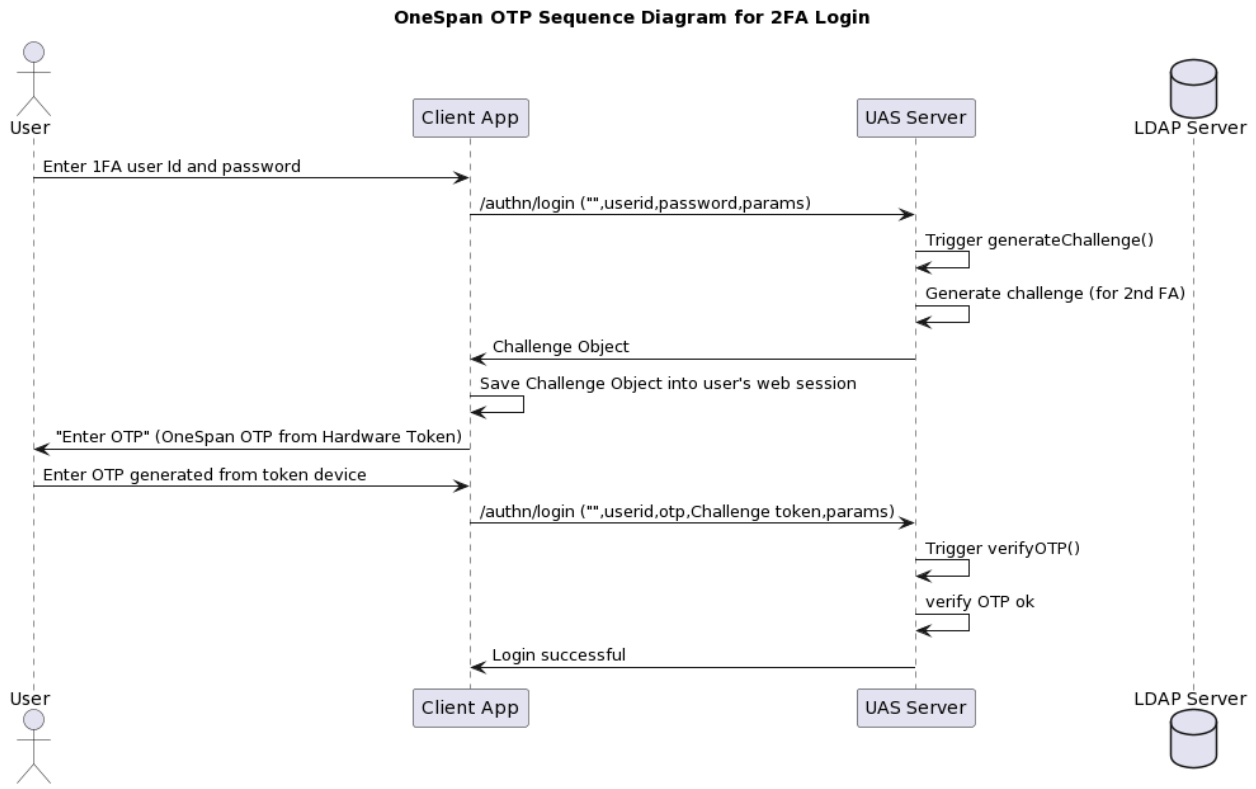
Login - OneSpan Token (OTP & Challenge Response) Module

- Use /authn/login to log in a user.
- Refer to the *AccessMatrix REST API* specification for more details.

Sequence Diagram

OneSpan OTP Sequence Diagram

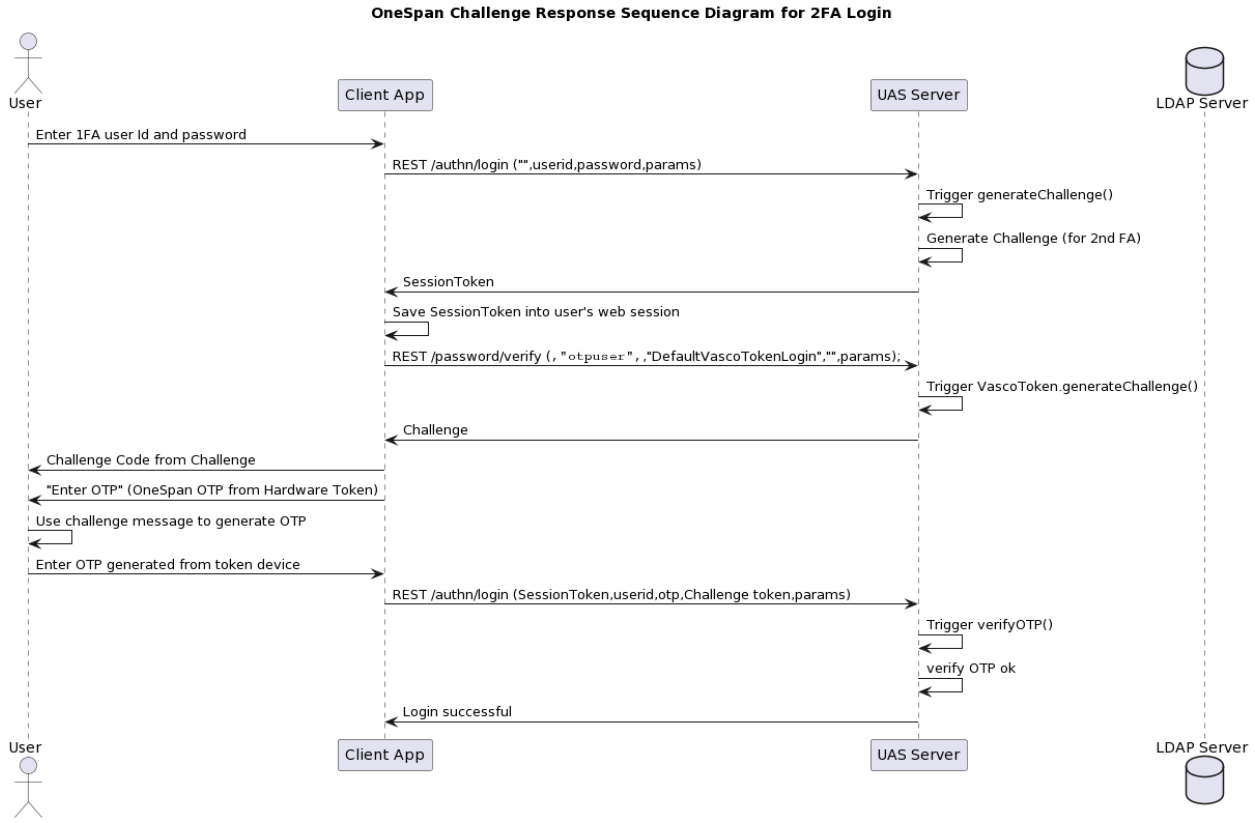
1. This section covers the two-factor authentication use case of OneSpan token OTP through the REST AuthenticationService Login URL /authn/login and also the Challenge Response feature.
2. The sequence diagram below shows the basic flow of the API calls for the OneSpan token OTP login.
3. By calling the REST /authn/login(sessionToken, userId, password, params) API, UAS will generate a challenge if the authentication realm passed in as the parameter is defined for VascoTokenLogin. The challenge is then used together with OTP from the token device for the 2nd factor login.
4. The user should pass in the authentication realm as realmId.
5. Invoke /authn/login(sessionToken, userId, password, challenge token, params) for the 2nd factor Login.
6. Password will be the OTP generated from the token device and challenge is from the 1st login call.
7. Authentication Realm should be passed in through params.



B. OneSpan Challenge Response Sequence Diagram

1. The sequence diagram below shows the basic flow of the API calls for the OneSpan challenge response login.
2. By calling the REST /authn/login(sessionToken, userId, password, challenge, params) URL, UAS will generate a challenge if the authentication realm passed in as the parameter is defined for VascoTokenLogin. The challenge code is then used to generate the OTP from the token device. Challenge + OTP generated are then used for 2nd factor Login.
3. The user should pass in the authentication realm as realmId.
4. User also need to pass in authMode in the JSON object params to define ChallengeResponse authMode ({"params":{"authMode":"CR"}}).
5. SessionToken returned from 1st login should be stored for use in 2nd login.
6. /password/verify(sessionToken, userId, userStoreId, loginModuleId, password, params) is called with password left as blank to generate the Challenge Object.
7. A challenge code and token will be returned together inside the Challenge Object.
8. Key in the challenge code in the token device to generate response OTP.
9. Invoke /authn/login(sessionToken, userId, password, challenge token, params) for 2nd factor login.

10. SessionToken will be the sessionToken from 1st login.
11. Password will be the OTP generated from the token device and the challenge is the challenge from /password/verify.
12. Authentication realm should be passed in through params.



REST Sample Code

A. OneSpan OTP REST API Sample

2FA Login API Calls
First Login Call • "VascoOtp2FA" is a sample realm with 2FA of system password basic + OneSpan token login • Password argument can be blank ("") if the Realm defined is only having OneSpan (VASCO) token login module • Password argument is mandatory if SystemPasswordBasic login module is used with OneSpan (VASCO) token login module for 2FA realm. • 1st Login will trigger a generateChallenge() at server side to generate ChallengeTO object and will be required for 2nd Login

2FA Login API Calls

URL: /authn/login

Request

JSON

```
{
  "sessionToken": "",
  "realmId": "VascoOtp2FA",
  "userId": "otpuser",
  "password": "password of first login module"
}
```

Response

JSON

```
{
  "result": {
    "sessionToken": "sessionToken returned in first call",
    "challenge": {
      "code": "...",
      "challengeToken": "challengeToken returned in first
call",

    }
  }
}
```

Second Login Call

- OTP from the OneSpan token device is passed in together with the challenge token from the 1st login to perform the 2nd login

URL: /authn/login

Request

JSON

```
{
  "sessionToken": "sessionToken returned in first call",
  "realmId": "VascoOtp2FA",
  "userId": "otpuser",
  "challengeToken": "challengeToken returned in first call",
  "password": "OTP generated by OneSpan token"
}
```

Response (on successful login)

JSON

2FA Login API Calls

```
{
  "result": {
    "sessionToken": "authenticated sessionToken should be
saved for successive calls",
    "authenticated": true,

  }
}
```

B. OneSpan Challenge Response REST Sample

2FA Login API Calls

First Login Call

- 1st FA login
  - Password argument can be blank (""), if the realm defined only has the OneSpan (Vasco) token login module
  - Password argument is mandatory if the SystemPasswordBasic login module is used with the OneSpan (Vasco) token login module for 2FA realm
  - Specify "authMode" for the challenge response "CR"
- Challenge Object contains the challenge code that needs to be entered into the token device for OTP generation

URL: /authn/login

Request

JSON

```
{
  "sessionToken": "",
  "realmId": "VascoCR2FA",
  "userId": "otpuser",
  "password": "password of first login module",
  "params": {
    "authMode": "CR"
  }
}
```

Response

JSON

```
{
  "result": {
    "sessionToken": "sessionToken returned in first call",
```

2FA Login API Calls

```
    "challenge": {
      "challengeToken": "challengeToken returned in first
call",
      "message": "....",

    }
  }
}
```

Second Login Call

- Key in the challenge message into the token device to generate a response

URL: /authn/login

Request

JSON

```
{
  "sessionToken": "sessionToken returned in first call",
  "challengeToken": "challengeToken returned in first call",
  "realmId": "VascoCR2FA",
  "userId": "otpuser",
  "password": "response generated by token device"

}
```

Response (on successful login)

JSON

```
{
  "result": {
    "sessionToken": "authenticated sessionToken should be
saved for successive calls",
    "authenticated": true,

  }
}
```

5.2. Token Verification

Generate OTP By Token Serial Number

- Use /token/generateOTP to generate an OTP.
- It can generate an OTP for a token device with software (virtual) modes, e.g. OneSpan virtual digipass).
- Typically, an event notification policy, a message yemplate and a delivery channel will be configured to handle the Generate OTP event to send out the OTP to the user.
- Refer to the *AccessMatrix REST API* specification for more details.

REST Sample Code

Notes:

- If you define the **From** field in the Message Template as ${event.Sender}, then you can pass the value from application side, e.g. "params":{"Sender": "isprint"}.
- If the user is not in the AccessMatrix system and it does not have the user's information, then you can define the value in **To** field of Message Template as ${event.MobilePhoneNo} instead of using $userMobile, and pass "params":{"MobilePhoneNo": "6581803157"}.
- To send out OTP via HTTP Delivery Channel, configure the **Event Notification Policy** attribute of the login module with the configured HTTP delivery channel.
- e.g. configure **Event Notification Policy** attribute of "DefaultVascoTokenLogin" login module with "HttpVDPOTP" delivery channel by passing "params":{"eventNotificationPolicyId": "HttpVDPOTP"}.
- You can also define the **From** field in Message Template as ${event.Sender} and pass the value from application side, e.g. "params":{"Sender": "eng@i-sprint.com"}.
- If the user is not in the AccessMatrix system and it does not have the user's information, then you can define the value in the **To** field of Message Template as ${event.Email} instead of using $userEmail, and pass "params":{"Email": "sandar@i-sprint.com"}.
- To send out OTP via Email Delivery Channel, configure the **Event Notification Policy** attribute of the login module with the configured Email delivery channel.
- e.g. configure **Event Notification Policy** attribute of "DefaultVascoTokenLogin" login module with "EmailVDPOTP" delivery channel by passing "params":{"eventNotificationPolicyId": "EmailVDPOTP"}.

- To return the OTP to client if policy allows (for demo only, usually it should not be returned), pass "params":{"returnOTPToClient": true}.

Generate OTP API Call

URL: /token/generateOTP

Request

JSON

```
{
  "sessionToken": "sessionToken of user with rights assign
token",

  "vendorToUnassign": "VASCO",
  "serialNumberToUnassign": "5510000006",

  "vendor": "VASCO",
  "serialNumber": "5510000014",
}
```

Response

- Response contains OTP because you passed "returnOTPToClient": true for demonstration.
- Normally, OTP should only be sent out to user and not returned to caller.

JSON

```
{
 "sessionToken": "10002TzCxnV3MuN......sK_oknvVpSJE",
 "policyId": "VascoKernelParm002"
 "vendor": "VASCO",
 "serialNumber": "0098310012",
 "params": {
    "eventNotificationPolicyId": "HttpVDPOTP",
    "MobilePhoneNo": "6581803157",
    "Sender": "isprint",
    "returnOTPToClient": true
  },
}
```

Verify OTP

- Use /token/user/assigned/list to retrieve the assigned tokens of a user based on the token vendor, token model, user store id and user Id.
- Use /token/verifyOTP to verify the OTP.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Get Assigned Tokens by User and Token Model API Call (REST)

URL: /token/user/assigned/list

Request (by user ID)

JSON

```
{
  "sessionToken": "session with rights to list assigned
tokens",
  "userId": "testuser",
  "vendor": "VASCO",
  "model": "DPSIM"
}
```

Request (by user UUID)

JSON

```
{
  "sessionToken": "session with rights to list assigned
tokens",
  "userId": "testuser",
  "vendor": "VASCO",
  "model": "DPSIM"
}
```

Response

JSON

```
{
  "result": [
    {
      "UUID": "....",
      "vendor": "VASCO",
      "serialNum": "....",
      "model": "DPSIM",

    },
  ]
}
```

Verify OTP By Serial Number API Call (REST)

URL: /token/verifyOTP

Request

JSON

Verify OTP By Serial Number API Call (REST)

```
{
  "sessionToken": "session with rights to verify OTP",
  "vendor": "VASCO",
  "serialNumber": "1000000014",
  "OTP": "903866"
}
```

Response

JSON

```
{
  "result": {
    "params": {
      "returnHostCode": "000",

    }
  }
}
```

Generate Challenge

- Use/token/generateChallenge to generate a challenge for the user to get a token response.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Generate Challenge API Call (REST)

URL: /token/generateChallenge

Request

JSON

```
{
  "dynamicAgentSession" : "true",
  "serialNumber" : "0097000002",
  "vendor" : "VASCO",
  "params" : ""
}
```

Response

JSON

```
{
    "result": {
```

Generate Challenge API Call (REST)


```
        "message": "2410",
        "challengeToken": "2410",
        "authenticationCode": -1,
        "params": {
            "tokenModel": "DP300",
            "challenge": "2410",
            "tokenSerNum": "0097000002",
            "tokenUUID": "4o7K8BuyL9",
            "tokenStatus": "Activated",
            "vendorId": "VASCO"
        },
        "code": "0"
    },
    "amProcessingTimeMillis": 382
}
```


Verify Response

- Use /token/verifyResponse to verify the response for a given challenge.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Verify Response API Call (REST)

URL: /token/verifyResponse

Request

JSON


```
{
  "dynamicAgentSession" : "true",
  "serialNumber" : "0097000001",
  "vendor" : "VASCO",
  "challenge" : "2410",
  "response" : "5902447"
}
```


Response

JSON


```
{
    "result": {
        "authenticationCode": -1,
        "params": {
            "tokenModel": "DP300",
            "tokenSerNum": "0097000001",
```

Verify Response API Call (REST)


```
            "tokenUUID": "AMwmakuyL8",
            "tokenStatus": "Activated",
            "vendorId": "VASCO"
        }
    },
    "amProcessingTimeMillis": 122
}
```


Verify Digital Signature

- Use /token/verifyDigitalSignature to verify the digital signature for a given input data.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Verify Digital Signature API Call (REST)

URL: /token/verifyDigitalSignature

Request

JSON


```
{
  "dynamicAgentSession" : "true",
  "serialNumber" : "FDT1306228",
  "vendor" : "VASCO_MULTI_DEVICE",
  "data" : [
    "32233223",
    ""
  ],
  "signature" : "753560",
  "params" : {
    "appId" : "1FLDSG    06",
    "authMode" : "SG"
  }
}
```


Response

JSON


```
{
  "result": {
    "authenticationCode": -1,
    "params": {
      "tokenModel": "DAL10",
      "deviceFriendlyName": "testDev01e",
      "tokenSerNum": "FDT1306228",
```

Verify Digital Signature API Call (REST)


```
      "sTimeWindow": 10,
      "isContextGroupScoreOK": false,
      "deviceType": "ROOTED_ANDROID",
      "tokenUUID": "IBrkMy-EHb",
      "isUserGroupScoreOK": false,
      "instanceNumber": "6",
      "tokenStatus": "Activated",
      "vendorId": "VASCO_MULTI_DEVICE",
      "sequenceNumber": 6,
      "isPlatformGroupScoreOK": false
    }
  },
  "amProcessingTimeMillis": 97
}
```

5.3. Token Management

Assign Token/ Unassign Token/ Reassign Token

- Use /token/assign to assign and/or unassign tokens to a holder (user or group typically).

  Note: This is useful for self-enrollment of a token by the user.

- For assigning a token, ensure the status of the token being assigned is "Activated" before making the token assignment.
- After unassigning the token, the status of the token being unassigned will be in "Unassigned" state.
- Use /token/reassign to reassign a token to a holder (user or group typically); basically reassign is a combined action of calling unassign token + assign token.
- After reassignment of a token, the status of the token being unassigned will be in "Assigned" state.

Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Assign Token/ Unassign Token/ Reassign Token API Call

URL: /token/reassign

Request

JSON

```
{
  "sessionToken": "sessionToken of user with rights assign
token",

  "vendorToUnassign": "VASCO",
  "serialNumberToUnassign": "5510000006",

  "vendor": "VASCO",
  "serialNumber": "5510000014",
}
```

Response

JSON

```
{
  "challenge": {
```

Assign Token/ Unassign Token/ Reassign Token API Call

```
    "params": {
      "assignedTokenUUIDs": [","],

    }
  }
}
```

Query Token Information

- Use /token/find to search for tokens based on given search criteria.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Query Token Information API Call (REST)

URL: /token/find

Request

JSON

```
{
 "sessionToken":
"10002BcdbJbg1W.........WLxy_qPx9SOdwgbvLIMTCEE78A4",
 "vendor_eq": "VASCO",
 "serialNum_eq": "0098310012",
 "model_eq": "DP300",
 "template": "properties"
}
```

Response

JSON

```
{
  "result": [{
    "objectClass": "Token",
    "vendor": "VASCO",
    "UUID": "C0A8012AA8D8011F82AE2E7F1B48A11A",
    "model": "DP300",
    "status": "Activated",
    "properties": [
      {
        "staticPinEnabled": "DP300",
        "tripleDes": "NO",
        "timeBasedAlgo": "YES",
        "appId": "APPL1       ",
        "staticPinDisabled": "000000",
```

Query Token Information API Call (REST)


```
      "responseChecksum": "NO",
      "lastEventValue": "0",
      "lastTimeUsed": "Thu Jan 01 00:00:00 1970",
      "maxInputFields": "0",
      "primaryTokenEnabled": "YES",
      "pinChForced": "NO",
      "eventValue": "0",
      "lastResponseType": "NA",
      "responseLength": "06",
      "pinChOn": "NO",
      "tokenModel": "DP300",
      "pinEnabled": "NO",
      "lastTimeShift": "0",
      "virtualTokenEnabled": "YES",
      "unlockSupported": "YES",
      "virtualTokenGracePeriodDate": "DISABLE",
      "timestepUsed": "000036",
      "virtualTokenType": "BACKUP",
      "useCount": "000000",
      "virtualTokenStatus": "3",
      "thresholdExceeded": "false",
      "responseType": "DEC",
      "codeWord": "00005200",
      "virtualTokenRemainUse": "-1",
      "pinLen": "0",
      "pinSupported": "NO",
      "errorCount": "0",
      "allowActivation": "",
      "model": "DP300",
      "eventBasedAlgo": "NO",
      "authMode": "RO",
      "pinMinLen": "0"
    },
    {
      "staticPinEnabled": "DP300",
      "tripleDes": "YES",
      "timeBasedAlgo": "NO",
      "appId": "APPL2       ",
      "staticPinDisabled": "000000",
      "responseChecksum": "NO",
      "lastEventValue": "0",
      "lastTimeUsed": "Thu Jan 01 00:00:00 1970",
      "maxInputFields": "3",
      "primaryTokenEnabled": "YES",
      "pinChForced": "NO",
      "eventValue": "0",
      "lastResponseType": "NA",
      "responseLength": "06",
      "pinChOn": "NO",
      "tokenModel": "DP300",
      "pinEnabled": "NO",
      "lastTimeShift": "0",
      "virtualTokenEnabled": "YES",
      "unlockSupported": "YES",
      "virtualTokenGracePeriodDate": "DISABLE",
      "timestepUsed": "000000",
      "virtualTokenType": "BACKUP",
```

Query Token Information API Call (REST)


```
        "useCount": "000000",
        "virtualTokenStatus": "3",
        "thresholdExceeded": "false",
        "responseType": "DEC",
        "codeWord": "00075000",
        "virtualTokenRemainUse": "-1",
        "pinLen": "0",
        "pinSupported": "NO",
        "errorCount": "0",
        "allowActivation": "",
        "model": "DP300",
        "eventBasedAlgo": "NO",
        "authMode": "SG",
        "pinMinLen": "0"
      },
      {
        "staticPinEnabled": "DP300",
        "tripleDes": "NO",
        "timeBasedAlgo": "NO",
        "appId": "APPL3       ",
        "staticPinDisabled": "000000",
        "responseChecksum": "YES",
        "lastEventValue": "0",
        "lastTimeUsed": "Thu Jan 01 00:00:00 1970",
        "maxInputFields": "1",
        "primaryTokenEnabled": "YES",
        "pinChForced": "NO",
        "eventValue": "0",
        "lastResponseType": "NA",
        "responseLength": "06",
        "pinChOn": "NO",
        "tokenModel": "DP300",
        "pinEnabled": "NO",
        "lastTimeShift": "0",
        "virtualTokenEnabled": "YES",
        "unlockSupported": "YES",
        "virtualTokenGracePeriodDate": "DISABLE",
        "timestepUsed": "000000",
        "virtualTokenType": "BACKUP",
        "useCount": "000000",
        "virtualTokenStatus": "3",
        "thresholdExceeded": "false",
        "responseType": "DEC",
        "codeWord": "000B08D0",
        "virtualTokenRemainUse": "-1",
        "pinLen": "0",
        "pinSupported": "NO",
        "errorCount": "0",
        "allowActivation": "",
        "model": "DP300",
        "eventBasedAlgo": "NO",
        "authMode": "CR",
        "pinMinLen": "0"
      }
    ],
    "name": "DP300 0098310012",
    "serialNum": "0098310012",
```

Query Token Information API Call (REST)


```
    "id": "VASCO-0098310012",
    "version": 29
  }],
  "amProcessingTimeMillis": 122
}
```


Change Token Status

- Use /token/changeStatus to change the token status.
    - By Serial Number: change the token status based on the token serial number.
    - By token UUID: change the token status based on the token UUID.
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Change Token Status Sample Request (REST)

URL: /token/changeStatus

Request (by token UUID)

JSON


```
{
 "sessionToken":
"10002IJ6IYf1kd2vx1aUi8yVWuBeynzW4g_vvohIsgb6d3pP4i9rdMihhty0pAvgNWfBXKCuGOy
So01owbQii0VxTHJZrrMqxX2mBwunIYoX1TXFkPYDSY39uMD139tKxngUpD0goQb6IALeVy3_7Qy
rm6e357ogPCbaxejlvdr2U9uclpeTz0rrTH3S3j_JY2L7nF9iT0i9sBh2TS5WeWAUGrV0TvW6tou
OM4-yvB5KjAFOECWq3W3EzZd0350xzIN9mrMVbZOv_UjbNvAuTTr17_8T-
9zUzXghzyf9j9zdm5GG6A1VqiHepycumLjPJO7Li2MQ_MnMgACQiSh5nZQ5CBUndvSz5WqapeMOz
Uglr_f1KRE6jbqfvRXDssgnxviV423hpymf4S9uioTuc9d_k9VfQwF4XjbP-_X8wb-do",
 "UUID": "C0A8012A9A29011E2461484A4396F53A",
 "status": "Activated"
}
```


Request (by Serial Number)

JSON


```
{
 "sessionToken":
"10002IJ6IYf1kd2vx1aUi8yVWuBeynzW4g_vvohIsgb6d3pP4i9rdMihhty0pAvgNWfBXKCuGOy
So01owbQii0VxTHJZrrMqxX2mBwuIYoX1TXFkPYDSY39uMD139tKxngUpD0goQb6IALeVy3_7Qyr
m6e357ogPCbaxejlvdr2U9uclpeTz0rrTH3S3j_JY2L7nF9iT0i9sBh2TS5WeWAUGrV0TvW6touO
M4-yvB5KjAFOECWq3W3EzZd0350xzIN9mrMVbZOv_UjbNvAuTTr17_8T-
9zUzXghzyf9j9zdm5GG6A1VqiHepycumLjPJO7Li2MQ_MnMgACQiSh5nZQ5CBUndvSz5WqapeMOz
Uglr_f1KRE6jbqfvRXDssgnxviV423hpymf4S9uioTuc9d_k9VfQwF4XjbP-_X8wb-do",
```

Change Token Status Sample Request (REST)

```
 "vendor":"VASCO",
 "serialNumber":"0091234568",
 "status": "Activated"
}
```

Response

JSON


```
{
  "result": {
    "responses": [{}],
    "challenge": {
      "params": {
        "tokenUUID": "C0A8012A9A29011E2461484A4396F53A",
        "oldTokenStatus": "Locked",
        "vendorId": "VASCO",
        "tokenModel": "DPGO1",
        "tokenStatus": "Activated",
        "tokenSerNum": "0091234568"
      },
      "authenticationCode": -1
    }
  },
  "amProcessingTimeMillis": 109
}
```


Parameters & Exceptions

Exceptions

Error Code	Description
30001	The token is not found for token UUID.
49999	Other reasons: No more tokens available/current token status is not assignable.
20000	Authorization fail, login user does not have the token admin role.

Resync Token

- Use /token/resync to resync the token.
- OTP token internal clock may drift (for environmental reasons, a big temperature difference or after a period of very long activity) and become invalid in the allowed time window for OTP verification. The Re-sync function needs to be called to erase all time synchronization

information, thus forcing the next authentication to be considered as the first one. Refer to the *VACMAN Controller Guide* for the details.

- Application needs to call a UAS API with parameters including token UUID, vendor id and token serial number. UAS will validate token Access id and token serial number against its database, and will return different error codes for "wrong token serial number" and "re-sync failure".
    - By serial number: **vendor** and **serialNumber** must be specified
    - By token UUID: **UUID** must be specified
- Refer to the *AccessMatrix REST API Specification* for more details.

REST Sample Code

Resync Token Sample Request (REST)

URL: /token/resync

Request (by token serial number)

JSON

```
{
 "sessionToken": "10002c5pRt........ATZq7z7u_3kdYS0L6g",
 "vendor": "VASCO",
 "serialNumber": "0091234568"
}
```

Request (by token UUID)

JSON

```
{
 "sessionToken": "10002c5pRt........ATZq7z7u_3kdYS0L6g",
 "UUID": "C0A8012A9A29011E2461484A4396F53A"
}
```

Response

JSON

```
{
  "result": {
    "responses": [],
    "challenge": {
      "params": {

      },
      "authenticationCode": 0
    }
  },
```

Resync Token Sample Request (REST)

```
  "amProcessingTimeMillis": 38
}
```

Parameters & Exceptions

Additional parameters:

Additional optional parameters for the JSON object params.

JSON

```
{
  "params": {
    "appId": "...",
    "newStatus": "...",

  }
}
```

Parameter	Description	Supported Values
appId	Token blob to resync	
newStatus	New token status after resync is done	

Exceptions:

Error Code	Description
30001	Token is not found for token UUID.
20000	Authorization fail, login user does not have the token admin role.

5.4. Errors

AccessMatrix REST API Error Codes, Faults or Exceptions

Authentication Faults and Error Codes

Refer to *Error Codes, Faults or Exceptions - "List of Authentication Faults/Error Codes"* in the *AccessMatrix REST API Specification*.

Authorization Faults and Error Codes

Refer to *Error Codes, Faults or Exceptions - "List of Authorization Faults/Error Codes"* in the *AccessMatrix REST API Specification*.

Query Faults and Error Codes

Refer to *Error Codes, Faults or Exceptions - "List of Query Faults/Error Codes"* in the *AccessMatrix REST API Specification*.

System (Runtime) Faults and Error Codes

Refer to *Error Codes, Faults or Exceptions - "List of System Faults/Error Codes"* in the *AccessMatrix REST API Specification*.

OneSpan Errors

Refer to *OneSpan VACMAN Controller Programmer's Guide* for the complete list of OneSpan error codes. This guide can be retrieved after the VACMAN Controller installation.

6. UAS OneSpan Token Housekeeping Policy

Housekeeping policy contains the rules on how to housekeep the OneSpan token data in the database. To view the housekeeping policy, navigate to **Administration Dashboard > System > Housekeeping** and click **Housekeeping Policy** on the left pane. Select the **UAS tab** and click the **Edit** button. Save the changes after modification.

Below is the list of OneSpan token-related housekeeping attributes:

Attribute Name	Description
UAS Tab	
Tokens	
Purge non-SMS token if it is in a certain status for longer than the specified period	Specifies how long non-SMS tokens can remain in a certain status before they are purged. Define the status for the token, i.e. "Battery Expired", and its period in months, i.e. "24", and then add the status and its period to the list. **Note:** Period (months) should be between 1 and 1200
Change non-SMS token status if it is not used for longer than the specified period	Specifies how long non-SMS tokens can remain unused before their current status is changed to a new status. Define the affected token group, the new status to change to, i.e. "Deactivated", and the time period for the tokens to remain unused before their status is changed to the new status. This attribute only affects tokens which meet the following requirements: • Token must be assigned to a user. • The current token status must have been defined as operational or usable. By default, this would be "Activated" and "Preactivation". The list of statuses can be configured using the attribute **Token Status for usable functions like verify OTP/signature/response etc.** For more information on how to configure this, refer to *UAS Administration Guide - Global Setting - Token Management*. Note: Period (months) should be between 1 and 1200.
Purge Token Instances in 'PreActivation' status older than (days)	Specifies how long (in days) token instances can remain in the "PreActivation" status before they are purged. **Note:** Value should be from 1 to 365
Purge 'Deactivated' Token Instances older than (days)	Specifies how long (in days) token instances can remain in the "Deactivated" status before they are purged. **Note:** Value should be from 1 to 365

7. Appendices

FAQ – How to Protect OneSpan Token Seeds

Token seed confidentiality in transit

Refer to *Vasco Controller Product Guide - Section 5.1 (DPX Import Service)*.
The transport key is used to protect Vasco token seeds in transit. The transport key is mailed separately from the DPX file to the token administrator. The customer can request more than one transport keys to be mailed to different token administrators. According to the Vasco (now renamed to OneSpan) documentation, the transport key is a 32-digit 3DES key.

Token seed confidentiality during import

AccessMatrix UAS automatically generates storage derive key (as defined by OneSpan) when a new token policy is created. The token policy must be created before tokens can be imported.

During the token import, AccessMatrix UAS passes in the transport key(s), storage-derive key and the DPX file to the Vasco Vacman Controller. The Vasco token BLOBs are returned to AccessMatrix UAS for storage. The BLOBs are protected by the storage derive key.

Token seed confidentiality in storage and in operation

The token BLOB (which includes the token seed) are protected by the storage derive key. During relevant token operations (e.g. OTP verification), AccessMatrix UAS calls the Vasco Vacman Controller and passes in the storage derive key together with the BLOB, along with other parameters.

The storage-derive key is stored as an attribute in AccessMatrix UAS and is protected by an Attribute Encryption Key (AEK). During the token policy creation, you can select the specific AEK key.
Each Encryption Key (AEK) is protected by either the Domain Master Key (DMK) or a long-term key from an HSM. (The DMK is a PBKDF2 derived key.)

Summary of keys used

Key Name	Cryptographic Algorithm	Purpose	Management
DPX Transport Key	3DES (as specified in Vasco documentation)	Protect the DPX files in transit	Sent by Vasco to the customer for DPX import (specific details must be requested from Vasco)
Storage Derive Key	Not available (specific details must be requested from Vasco)	Protect the token BLOBs in storage; during token operation, the key is sent to Vacman Controller	Auto-generated (when the token policy is created) by AccessMatrix based on Vasco specification
Attribute Encryption Key (AEK)	AES	Protect the Storage Derive Key	Auto-generated (when AEK is created) by AccessMatrix; can select to be protected by DMK or a key in HSM
HSM Key	AES	Protect AEK	Managed at the HSM; must be backed-up
Domain Master Key (DMK)	AES	Protect AEK and other encryption keys	Derived using PBKDF2 with proprietary secret